

Kalman Linear Attention: Parallel Bayesian Filtering For Efficient Language Modeling and State Tracking

Vaisakh Shaj¹ Cameron Barker¹ Aidan Scannell¹ Andras Szecsenyi¹ Elliot J. Crowley¹ Amos Storkey¹

Abstract

State-space language models such as Mamba and gated linear attention (GLA) offer linear-complexity, parallelisable alternatives to transformers, but often lack the expressivity and robust state-tracking needed for complex reasoning. We address this by reframing sequence modeling through a probabilistic lens, with Bayesian filters as the core primitive. Classical Kalman filters provide principled state and uncertainty estimation but are typically viewed as inherently sequential. We show that reparameterising the Kalman filter in information form casts its updates as an associative scan, enabling efficient parallel training. The resulting Kalman Linear Attention (KLA) layer performs time-parallel probabilistic inference while maintaining explicit belief-state uncertainty, offering strictly more expressive nonlinear updates and gating than GLA variants while retaining their computational advantages. On language modeling tasks, KLA matches or outperforms modern SSMs and GLAs on discrete token-manipulation, state-tracking, and zero-shot commonsense-reasoning benchmarks, and is among the first stacked Bayesian-filtering primitives pretrained at the billion-token scale.

1. Introduction

Scaling sequence modeling beyond quadratic attention (Vaswani et al., 2017) is a central goal in large language and foundation model research.¹ This has motivated interest in RNN-like architectures that support linear-time sequence modeling with efficient parallel training. Recent state-space models (SSMs), including S4/S5 (Gu et al., 2021; Smith et al.,

¹University of Edinburgh, Edinburgh, United Kingdom. Correspondence to: Vaisakh Shaj <vaisakhs.shaj@gmail.com>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹Code: github.com/vaisakh-shaj/kalman-linear-attention

	Softmax Attention	SSMs / GLA	KLA
Expressivity	Nonlinear	Linear	Fractional lin. (Möbius)
Training eff.	$\mathcal{O}(T^2)$	$\mathcal{O}(T)$	$\mathcal{O}(T)$
Inference eff.	$\mathcal{O}(T)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Seq. uncertainty	✗	✗	✓
Parallel training	✓	✓	✓

Table 1. High-level comparison of sequence-mixing primitives.

2022), Mamba (Gu & Dao, 2023; Dao & Gu, 2024), and their successors, achieve this with $\mathcal{O}(\log T)$ depth, $\mathcal{O}(T)$ work, and linear or sublinear memory. This is essential for handling long contexts, on-device deployment and energy efficiency.

Despite their efficiency, adoption of SSMs is not as widespread as attention-based transformers. Recent work reinterprets modern SSMs and linear RNNs, including Mamba, as gated variants of linear attention (GLA) (Yang et al., 2023), where performance largely depends on how gates are defined. In all of these, the underlying hidden state updates are linear or affine, which limits their expressivity compared to softmax attention, whose normalisation induces nonlinear interactions between tokens. Furthermore, none of these represent state uncertainty explicitly. This contrasts with longstanding probabilistic linear state-space formalisms that lead to Kalman filter updates. These have both state uncertainty and nonlinear parameter updates.

In this paper, we ask: *Can we overcome the linear update constraints common to current GLA models to develop a state-space block that efficiently implements Kalman-like updates?*

We introduce **Kalman Linear Attention (KLA)**, which formulates sequence modeling as a Bayesian filtering problem. KLA models two sources of uncertainty: process noise, which captures uncertainty in state evolution, and observation noise, which captures uncertainty in the information provided by each input token. Crucially, this uncertainty is not merely an output: it directly controls how new information is gated in state spaces.

At first glance, such Bayesian updates seem inherently sequential. Our key insight is that the information form of the Kalman filter admits a fractional-linear / Möbius associative structure that can be implemented using parallel scan-based algorithms, *despite its nonlinear update computation*. This allows Kalman-style updates to achieve the same $\mathcal{O}(\log T)$

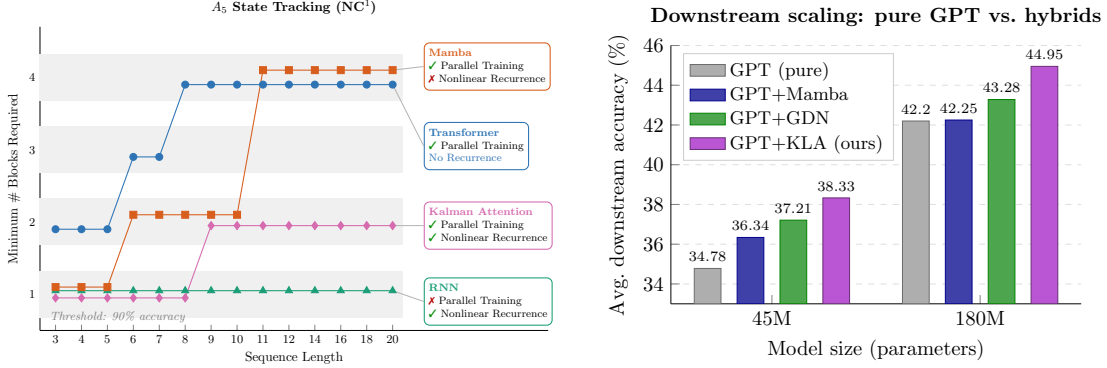


Figure 1. **(a) Expressivity.** Minimum number of blocks required to solve the A_5 (alternating group on 5 elements) permutation-composition task (Merrill et al., 2024). KLA’s fractional-linear (Möbius) updates fall between a fully nonlinear RNN and linear SSMs/transformers, solving the task at constant depth (1-2 blocks) where linear SSMs/attention require depth growing with sequence length, while remaining parallel-trainable. **(b) Downstream scaling.** Zero-shot accuracy (averaged over eight commonsense benchmarks) of softmax-attention GPT vs. GPT+SSM hybrids at 45M and 180M parameters. Replacing a single final attention layer with KLA (GPT+KLA) gives the strongest complement to attention.

parallel depth as models such as Mamba. The result is the **KLA** layer: a drop-in replacement for standard SSM or attention layers.

It is not enough to implement a new primitive without establishing that the extra expressiveness provides a benefit. We study the suitability of this continuous-state probabilistic model for discrete language modeling both theoretically and empirically, demonstrating important gains.

Contributions. (C1) **Associative reparameterisation of Kalman filtering:** we reparameterise the diagonal linear-Gaussian filter in information form, showing the precision recursion is a Möbius (fractional-linear) map that composes associatively, enabling parallel prefix scans. (C2) **Nonlinear gating from uncertainty:** the precision-ratio gates are history-dependent and nonlinear, going beyond linear/affine SSM and gated linear-attention updates while preserving linear-time scan structure. (C3) **Kalman Linear Attention layer:** we introduce KLA, a drop-in sequence mixer for modern language-modeling pipelines that produces explicit belief-state uncertainty. (C4) **Scaling and empirical validation:** our scan-based implementation scales efficiently with sequence length and matches or outperforms modern SSMs and GLAs on state-tracking, associative recall, and zero-shot commonsense reasoning, and is among the first stacked Bayesian filters pretrained at the billion-token scale.

Conflict of Interest Disclosure. The authors declare no conflicts of interest; this is purely academic work.

2. Related Work

Attention as filtering. The idea that intelligent systems must filter information predates modern self-attention: in cognitive science, early theories framed attention as a mechanism that suppresses irrelevant signals and routes

task-relevant ones under limited processing capacity (Broadbent, 1957; Treisman, 1969; Deutsch & Deutsch, 1963; Nielsen & Garcia, 2009). The core theme across these views is that attention allocates scarce processing by selecting what matters in context.

In modern language models, transformer self-attention can be viewed as a form of context-based filtering (Vaswani et al., 2017). Tokens compete via similarity and the softmax suppresses most inputs whilst allowing a small subset to dominate the update. But this selection is *exemplar-based*: it requires retaining and comparing against all past keys/values, largely eliminating an explicit bottleneck and yielding $\mathcal{O}(T^2)$ cost at long context lengths.

Bayesian filtering and precision-control as attention. Bayesian filters perform filtering as posterior inference: prior beliefs are updated with new observations weighted by their precision (inverse variance), so more reliable observations exert greater influence whilst unreliable evidence is down-weighted. This yields a principled, interpretable gating mechanism where selection follows from uncertainty propagation.

Precision-weighted prediction errors have also been proposed as a computational account of attention in neuroscience and predictive coding (Rao & Ballard, 1999; Feldman & Friston, 2010). In this view, attention modulation is based upon confidence in sensory signals as opposed to deterministic reweighting. We adopt this view and show that Bayesian filters can be integrated into modern sequence modeling.

Temporal parallelisation of Bayesian filters. Särkkä & García-Fernández (2021) pioneered temporal parallelisation of Bayesian filtering and smoothing using prefix-sum algorithms, requiring specialised parameterisations to ensure associativity. To achieve associativity, they lift each filtering step into a 5-tuple augmented representation (linear case). We show that no such augmentation is needed: Kalman up-

dates in the information form are already Möbius maps, so precision composes by 2×2 matrix multiplication, a minimal, GPU-friendly structure (Theorems 1 and 2). Beyond parallelisation, we integrate these probabilistic primitives into learnable neural sequence models for language, where token-dependent observation likelihoods and dynamics are learned from data rather than specified a priori.

Deep Kalman filters and probabilistic state-space models. Kalman filtering (Kalman, 1960) has a long history in RL, control, and world modeling (Haarnoja et al., 2017; Watter et al., 2015; Hafner et al., 2019; 2020; Shaj et al., 2023). Deep Kalman filters embed linear-Gaussian SSMs in neural networks, trained by exact or variational inference (Krishnan et al., 2015; 2017; Karl et al., 2017; Fraccaro et al., 2016; Becker et al., 2019; Shaj et al., 2021), but their recurrent update limits scalability to long contexts. Recent deep-RL work pairs deterministic SSM backbones with Kalman-style dynamics via Särkkä’s temporal parallelisation (Becker et al., 2024), or replaces them with parallelised Kalman layers for uncertainty under partial observability (Luis et al., 2024). In contrast, we adopt KLA as a *standalone*, closed-form probabilistic primitive for language modeling, competing directly with modern SSM/GLA mixers without an extra deterministic scaffold. Crucially, whereas these prior approaches typically employ a single Kalman layer, we hierarchically stack KLA across many layers, training deep multi-layer Bayesian-filtering architectures for language modeling at the billion-token scale.

3. Background and Preliminaries

Modern language models compose a shared representation map $f(\cdot; \theta)$ and a task-specific head g (Alonso et al., 2025): given a prompt $x_{1:T}$, the backbone produces latents $z_{1:T} = f(x_{1:T}; \theta)$ and the head maps them to outputs $y_{1:T}$ (e.g., next-token prediction, tagging, regression). Concrete parameterisations for f balance expressivity with computational tractability: transformers use softmax attention as the core primitive, whereas state-space models (SSMs) such as Mamba adopt structured recurrence with linear complexity.

Deterministic state-space models (SSMs) and Mamba. Popular SSMs such as Mamba originate from structured state-space models (Gu et al., 2021). Building on their diagonal simplification (Gupta et al., 2022), a linear SSM maps a scalar input signal $x(t)$ to an output $y(t)$ through a latent state $h(t) \in \mathbb{R}^N$:

$$h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t), \quad (1)$$

with $x(t), y(t) \in \mathbb{R}$, $\mathbf{A} = \text{diag}(a_1, \dots, a_N)$, $\mathbf{B} \in \mathbb{R}^N$, and $\mathbf{C} \in \mathbb{R}^N$. After discretisation, one processes a D -dimensional sequence $\mathbf{X} \in \mathbb{R}^{T \times D}$ by running D independent copies of this recurrence, one per feature coordinate, or channel-, each with its own scalar input $x(t)$.

Mamba (S6) (Gu & Dao, 2023) makes this recurrence input-dependent (selective). For a single channel the update is

$$\mathbf{h}_t = \bar{\mathbf{a}}_t \odot \mathbf{h}_{t-1} + \bar{\mathbf{b}}_t x_t, \quad y_t = \mathbf{c}_t^\top \mathbf{h}_t, \quad (2)$$

where $x_t, y_t \in \mathbb{R}$; this system can be applied independently to each feature channel of a D -dimensional sequence. Here $\mathbf{h}_t \in \mathbb{R}^N$ is the latent state, $\bar{\mathbf{a}}_t, \bar{\mathbf{b}}_t \in \mathbb{R}^N$ are discretised diagonal dynamics, and $\mathbf{c}_t \in \mathbb{R}^N$ is an input-dependent readout (the selective counterpart of $\mathbf{C}$ in Equation (1)). The key property is *selectivity*: all parameters $(\bar{\mathbf{a}}_t, \bar{\mathbf{b}}_t, \mathbf{c}_t)$ are computed from the current input x_t , allowing the model to selectively filter which information to retain or discard at each step, while preserving linear-time scanning. This input-dependent gating can be seen as a learned form of filtering; in Section 3 we formalise this intuition through Bayesian filtering with explicit uncertainty.

State expansion. The dimension N in Equation (2) is a design choice known as *state expansion* (Gupta et al., 2022; Poli et al., 2024; Yu & Erichson, 2025). With $N=1$, each feature channel maintains a single scalar recurrence and the state is $\mathbf{h}_t \in \mathbb{R}^D$. Setting $N>1$ equips every channel with N parallel memory slots, expanding the state to $\mathbf{H}_t \in \mathbb{R}^{N \times D}$. Stacking the per-channel dynamics accordingly, we write $\bar{\mathbf{A}}_t, \bar{\mathbf{B}}_t \in \mathbb{R}^{N \times D}$ and $\mathbf{C}_t \in \mathbb{R}^{1 \times N}$. How the dynamics are allocated across these slots is itself an algorithmic decision: $\bar{\mathbf{A}}_t$ and $\bar{\mathbf{B}}_t$ may be *distinct* per slot or *shared* (tied) across channels, depending on the model variant. State expansion trades a larger memory footprint ($N \times D$ vs. D) for richer per-channel recurrence histories, as each slot can capture a different effective time-scale.

Attention from the SSM perspective. Ignoring scaling constants, auto-regressive softmax self-attention computes

$$\mathbf{Y} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top + \mathbf{M})\mathbf{V}, \quad (3)$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{X}\mathbf{W}_q, \mathbf{X}\mathbf{W}_k, \mathbf{X}\mathbf{W}_v$ for $\mathbf{X}, \mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{T \times D}$, $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v \in \mathbb{R}^{D \times D}$ and $\mathbf{M} \in (-\infty, 1)^{T \times T}$ is an auto-regressive mask.

Unlike the linear recurrence in Equation (2), softmax attention maintains a distinct memory of each $\mathbf{K}_t$ and $\mathbf{V}_t$, yielding $\mathcal{O}(T^2)$ time and memory complexity.

By removing softmax we obtain Linear Attention $y_t = \mathbf{q}_t \sum_{i=0}^t \mathbf{k}_i^\top \mathbf{v}_i$ which permits an alternate recurrent form (Katharopoulos et al., 2020) that can be calculated in $\mathcal{O}(T)$ time and $\mathcal{O}(1)$ memory complexity, given by

$$\mathbf{H}_t = \mathbf{G}_t \odot \mathbf{H}_{t-1} + \mathbf{k}_t^\top \mathbf{v}_t \quad \text{and} \quad \mathbf{y}_t = \mathbf{q}_t \mathbf{H}_t. \quad (4)$$

where $\mathbf{G}_t \in \mathbb{R}^{N \times D}$ is an optional gating term.

Identifying $\mathbf{G}_t \equiv \bar{\mathbf{A}}_t$, $\mathbf{k}_t^\top \equiv \bar{\mathbf{B}}_t$ and $\mathbf{q}_t \equiv \mathbf{C}_t$ shows that GLA (Yang et al., 2023) matches Mamba when $\mathbf{W}_v = \mathbf{I}$.

Gaussian state-space models and Bayesian filtering. A classical probabilistic lens on sequential data is *state estimation*: we posit an (unobserved) latent state $\mathbf{z}_t \in \mathbb{R}^d$ that evolves over time, and assume the observed sequence $\mathbf{o}_t \in \mathbb{R}^p$ provides noisy evidence about that state. The goal of *Bayesian filtering* is to maintain the posterior belief $p(\mathbf{z}_t \mid \mathbf{o}_{1:t})$ online, recursively combining a *dynamics prior* (predict) with an *observation likelihood* (update).

When the dynamics and observations are linear with Gaussian noise, we obtain a linear-Gaussian state-space model:

$$\mathbf{z}_t = \mathbf{A}_t \mathbf{z}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_t), \quad (5)$$

$$\mathbf{o}_t = \mathbf{C}_t \mathbf{z}_t + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, \Sigma_t^{\text{obs}}), \quad (6)$$

in which the filtering posterior remains Gaussian, $p(\mathbf{z}_t \mid \mathbf{o}_{1:t}) = \mathcal{N}(\boldsymbol{\mu}_t, \Sigma_t)$ and can be computed in closed form via the *Kalman filter* (KF), which is optimal in the minimum mean-squared error (MMSE) sense under these assumptions.

Control versus observation: two ways to read a token sequence. This filtering view contrasts with the common “controlled dynamics” interpretation of deterministic mixers (e.g. SSM/CDE-style models) (Muca Cirone et al., 2024), where the input is an exogenous control driving the hidden state forward; Bayesian filtering instead treats it as noisy observations of a stochastic latent process. This swap, control versus observation, changes what is random and what computation is appropriate, forward simulation versus posterior inference (Figure 6). We adopt the filtering perspective throughout, yielding an attention-like mixer with explicit uncertainty propagation.

Notational conventions. We index token sequences by $t \in \{1, \dots, T\}$. Bold uppercase denotes matrices (or collections over time), bold lowercase vectors/sequences (e.g. $\mathbf{x}_t$), and lowercase scalars; $\odot$ and $\oslash$ (equivalently the fraction bar $\mathbf{A}/\mathbf{B}$) denote the elementwise (Hadamard) product and division of equal-shape arrays, and $(\cdot)^{-1}$ on a vector is elementwise. We write d_{model} for the model width (the residual-stream / token-embedding dimension), N for the state-expansion factor (the per-channel latent size, called d_{state} in SSM codebases), and D for the value (head) dimension, so the state-expanded belief state is $\mathbf{S}_t \in \mathbb{R}^{N \times D}$. In the diagonal (per-coordinate) parameterisation, we identify diagonal matrices with their diagonal terms and apply scalar recursions elementwise. An overbar denotes the discretised counterpart of a continuous-time parameter, e.g. $\bar{a}_t$ is obtained by discretising a continuous-time decay parameter.

Appendix D provides additional background material on related mathematical constructs.

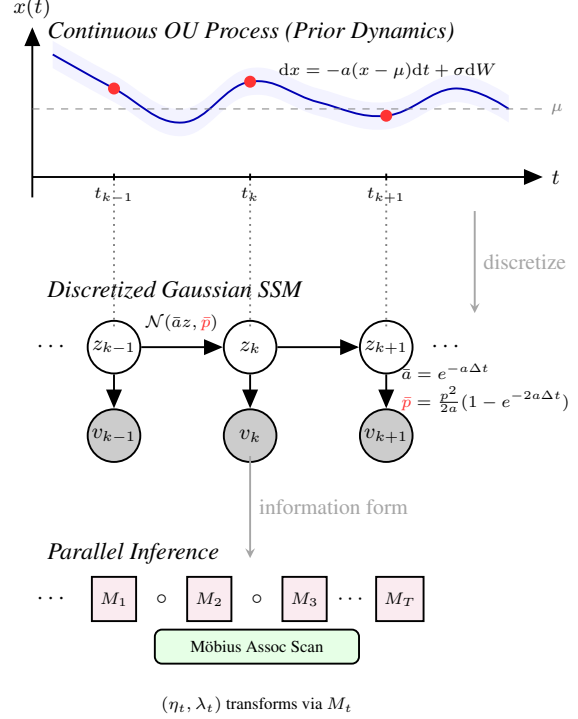


Figure 2. From OU dynamics to parallel inference. **(Top)** Continuous-time OU prior. **(Middle)** Discrete linear-Gaussian SSM. **(Bottom)** Möbius scan for parallel posterior state estimation.

4. Method

We introduce *Kalman Linear Attention (KLA)* as a *probabilistic* sequence mixer. Instead of deterministically updating a hidden state as in modern SSM mixers, KLA maintains a belief state over a latent representation, consisting of a posterior mean and an explicit uncertainty (precision/covariance). This probabilistic view turns sequence mixing into state estimation from noisy token evidence, and provides a principled mechanism for adaptivity and a gating mechanism derived from relative uncertainties of state and observation.

Our method is organised around two ideas. **(i) A probabilistic SSM view of sequence mixing.** We cast token processing as inference in a linear-Gaussian state-space model. Each token contributes a (noisy) observation v_t of the latent state through an observation operator k_t , while the latent evolves under a continuous-time stochastic prior, discretised for sequences.

(ii) Parallelisable filtering via information form. We address the main obstacle to using Bayesian filters as sequence mixers: classical filtering is recursive and appears inherently sequential. We show that by working in *information form* (precisions and natural parameters), the posterior updates acquire a parallelisation-friendly Möbius (fractional-linear) structure. This yields a scalable Bayesian sequence mixer with the computational profile of modern linear-time mixers, while preserving clear probabilistic semantics.

Notation and scope. Throughout, **blue terms** denote uncertainty-aware quantities absent in deterministic SSMs. For readability, the main text fixes the state-expansion factor to $N=1$ and presents the *diagonal* (per-channel) Kalman filter, so that every recursion reduces to scalar (elementwise) operations; the state-expanded ($N>1$) matrix form generalises this directly (the state updates of Table 2 and Algorithm 1).

4.1. Stochastic Dynamics and Token Likelihood

Dynamics prior (predict step) via OU discretisation. Modern SSM mixers (Gu et al., 2021; Smith et al., 2022) are typically motivated from continuous-time dynamics and discretised to operate on tokens: a simple linear ODE yields a discrete recurrence whose decay $\bar{\mathbf{a}}_t$ acts as a forget factor and step size Δt sets the time scale, casting SSM channels as a spectrum of memories indexed by decay rate.

KLA keeps this continuous-to-discrete pipeline but makes one change: we treat latent evolution as inherently uncertain, specifying a continuous-time *stochastic* prior and discretising it exactly. We use an Ornstein-Uhlenbeck (OU) process, the canonical mean-reverting diffusion (continuous-time analogue of a stable AR(1)), which preserves exponential forgetting while adding an explicit process-noise term capturing unmodelled variability in the latent dynamics. Under exact discretisation, the OU prior induces a Gaussian transition

$$\mathbf{z}_t \mid \mathbf{z}_{t-1} \sim \mathcal{N}(\bar{\mathbf{a}}_t \odot \mathbf{z}_{t-1}, \bar{\mathbf{p}}_t), \quad (7)$$

$$\bar{\mathbf{a}}_t = e^{-\mathbf{a} \Delta t}, \quad \bar{\mathbf{p}}_t = \frac{\mathbf{p}^2}{2\mathbf{a}} \odot (1 - e^{-2\mathbf{a} \Delta t}), \quad (8)$$

where $\bar{\mathbf{a}}_t$ denotes the decay and $\bar{\mathbf{p}}_t$ the process-noise variance.

Multi-channel specialisation. By construction $\bar{\mathbf{p}}_t$ is coupled to $\bar{\mathbf{a}}_t$: the same parameters $\mathbf{a}$ that set decay also govern how uncertainty accumulates between observations, giving per-channel specialisation along two linked axes, memory decay and drift. Different coordinates learn different decay rates (short- vs. long-memory channels), with process noise $\bar{\mathbf{p}}_t$ setting how far each channel may drift between tokens. Unlike Mamba, whose token-dependent timescale Δ_t doubles as the selection/filtering mechanism, we keep the dynamics parameters *time-invariant* and let selection emerge purely from the uncertainties, freeing Δ_t to specialise solely in per-channel time-scales.

Input token likelihood as noisy evidence (update step). To connect this dynamics prior to input token sequences, we model each token as providing noisy evidence about the latent state:

$$\mathbf{v}_t \mid \mathbf{z}_t \sim \mathcal{N}(\mathbf{k}_t \odot \mathbf{z}_t, (\Lambda_t^v)^{-1}), \quad (9)$$

where $\mathbf{v}_t$ is token-derived content, $\mathbf{k}_t$ is an observation operator, and Λ_t^v is the value precision representing confidence

in the token evidence. Combining Equation (7) with Equation (9) yields a linear-Gaussian state-space model, and *sequence mixing corresponds to posterior inference* of the latent state given token evidence.

Gaussian SSM view of q - k - v interactions. We can summarise the attention-aligned probabilistic interpretation as a linear-Gaussian state-space model in which keys and values parameterise the *likelihood* (token evidence), while queries parameterise a *readout* applied after inference. Let $\mathbf{z}_t$ denote the latent state and let $(\mathbf{k}_t, \mathbf{v}_t, \Lambda_t^v)$ be token-dependent likelihood parameters produced from the input sequence. The generative model is:

$$\textbf{Prior} \quad \mathbf{z}_t \mid \mathbf{z}_{t-1} \sim \mathcal{N}(\bar{\mathbf{a}}_t \odot \mathbf{z}_{t-1}, \bar{\mathbf{p}}_t), \quad (10)$$

$$\textbf{Evidence} \quad \mathbf{v}_t \mid \mathbf{z}_t \sim \mathcal{N}(\mathbf{k}_t \odot \mathbf{z}_t, (\Lambda_t^v)^{-1}), \quad (11)$$

$$\textbf{Readout} \quad \mathbf{y}_t \mid \mathbf{z}_t \sim \mathcal{N}(\mathbf{q}_t \odot \mathbf{z}_t, (\Lambda_t^{\text{out}})^{-1}). \quad (12)$$

In this view, $\mathbf{k}_t$ specifies the observation geometry, $\mathbf{v}_t$ the token evidence, and Λ_t^v its reliability (precision), while the query $\mathbf{q}_t$ selects what component of the latent is exposed. The scan runs in information form $(\boldsymbol{\eta}_t, \boldsymbol{\lambda}_t)$, with $\boldsymbol{\eta}_t$ the accumulated precision-weighted evidence and $\boldsymbol{\lambda}_t$ the total precision; at readout we expose the posterior mean $\mathbf{y}_t = \mathbf{q}_t \odot \boldsymbol{\mu}_t$, $\boldsymbol{\mu}_t = \boldsymbol{\eta}_t \odot \boldsymbol{\lambda}_t$. The division by $\boldsymbol{\lambda}_t$ is a genuine normalisation: $\boldsymbol{\eta}_t$ is an unnormalised numerator (content scaled by accumulated confidence), whereas $\boldsymbol{\mu}_t$ is its precision-weighted average, on the same scale as the values - the analogue of dividing attention’s evidence $\sum_i e^{s_i} \mathbf{v}_i$ by its normaliser $\sum_i e^{s_i}$ (so the readout is an average, not an accumulating sum) and of the $\mathbf{C}_t$ -projection $\mathbf{y}_t = \mathbf{C}_t \odot \mathbf{h}_t$ in deterministic SSMs, with the precision $\boldsymbol{\lambda}_t$ exposed separately as uncertainty. This is the deterministic-readout limit $\Lambda_t^{\text{out}} \rightarrow \infty$ of equation 12; reading the mean is the natural, well-scaled choice rather than a necessity.

4.2. Parallel Inference via Möbius and Affine Scans

A major obstacle to using Bayesian filters as sequence mixers is that classical filtering is *recursive* and thus appears inherently sequential. Our key observation is that, in *information form*, the updates admit a *compositional* structure that can be implemented with parallel scans.

Information form. We represent the Gaussian posterior by its precision and natural (canonical) parameter:

$$p(\mathbf{z}_t \mid \mathbf{v}_{1:t}) = \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\lambda}_t^{-1}), \quad \boldsymbol{\eta}_t := \boldsymbol{\lambda}_t \odot \boldsymbol{\mu}_t. \quad (13)$$

This parameterisation is convenient because, in linear-Gaussian models, incorporating token evidence corresponds to *adding* canonical parameters (precisions and information means), while the remaining “predict” transformation induced by the dynamics takes a structured form. As we show next, these structured updates *compose associatively* across time, which is exactly what enables parallel prefix scans.

Theorem 1 (Precision Update as a Möbius Transformation). *Let λ_t be the posterior precision at time t in the diagonal linear-Gaussian model Equation (7)-Equation (9). Define $\phi_t := \mathbf{k}_t^2 \odot \Lambda_t^\vee$. Then the map $\lambda_{t-1} \mapsto \lambda_t$ is a linear-fractional (Möbius) transformation:*

$$\lambda_t = \mathbf{M}_t(\lambda_{t-1}) = \frac{\alpha_t \odot \lambda_{t-1} + \beta_t}{\gamma_t \odot \lambda_{t-1} + \delta_t}, \quad (14)$$

$$\mathbf{M}_t = \begin{pmatrix} \alpha_t & \beta_t \\ \gamma_t & \delta_t \end{pmatrix} = \begin{pmatrix} 1 + \bar{\mathbf{p}}_t \odot \phi_t & \bar{\mathbf{a}}_t^2 \odot \phi_t \\ \bar{\mathbf{p}}_t & \bar{\mathbf{a}}_t^2 \end{pmatrix}. \quad (15)$$

Interpretation: precision track as uncertainty-driven gating. While Equation (14) is nonlinear in λ_{t-1} , it admits a simple “gate-like” rearrangement:

$$\lambda_t = \underbrace{(\bar{\mathbf{a}}_t^2 + \bar{\mathbf{p}}_t \odot \lambda_{t-1})^{-1} \odot \lambda_{t-1}}_{(\text{confidence history})} + \underbrace{\phi_t = \mathbf{k}_t^2 \odot \Lambda_t^\vee}_{(\text{confidence in current token})}. \quad (16)$$

The shared denominator $\bar{\mathbf{a}}_t^2 + \bar{\mathbf{p}}_t \odot \lambda_{t-1}$ introduces *history dependence*: as accumulated precision grows, the model naturally becomes more selective about incorporating new evidence. We name its inverse $\rho_t := \mathbf{1} \odot (\bar{\mathbf{a}}_t^2 + \bar{\mathbf{p}}_t \odot \lambda_{t-1})$, so the history term of Equation (16) is $\rho_t \odot \lambda_{t-1}$. The *same* factor ρ_t reappears in the mean recursion below as the forget gate $\mathbf{f}_t = \rho_t \odot \bar{\mathbf{a}}_t$ (Theorem 2), tying the precision and mean tracks together and inducing the input/forget-gate behaviour of the mean update (Equation (17)).

Corollary 1 (Precision Updates via Parallel Prefix Scan). *Given λ_0 and matrices $\{\mathbf{M}_t\}_{t=1}^T$ from Theorem 1, the posterior precisions $\{\lambda_t\}_{t=1}^T$ can be computed by a parallel prefix scan over $\{\mathbf{M}_t\}$ with $\mathcal{O}(T)$ work and $\mathcal{O}(\log T)$ depth.*

Theorem 2 (Mean Update as Affine Transformations). *Let $\eta_t := \lambda_t \odot \mu_t$ be the posterior information mean and $\Lambda_t^\vee$ the value precision. Given the precision path $\{\lambda_t\}$, the information mean evolves affinely:*

$$\eta_t = \underbrace{(\bar{\mathbf{a}}_t^2 + \bar{\mathbf{p}}_t \odot \lambda_{t-1})^{-1} \odot \bar{\mathbf{a}}_t}_{\mathbf{f}_t \text{ (history-dependent forget gate)}} \odot \eta_{t-1} + \underbrace{\mathbf{k}_t \odot \Lambda_t^\vee \odot \mathbf{v}_t}_{\text{token evidence}}. \quad (17)$$

Corollary 2 (Mean Updates via Parallel Prefix Scan). *The posterior information means $\{\eta_t\}_{t=1}^T$ are computable via parallel prefix scan over the affine transformations in Theorem 2 in $\mathcal{O}(T)$ work and $\mathcal{O}(\log T)$ depth.*

Special cases: Under deterministic ($\mathbf{p}_t = 0$) and linear time-invariant (LTI) settings, the KLA updates reduce to convolutions computable in $\mathcal{O}(\log T)$ time (Theorem 3).

KLA as a gated RNN: gates from Bayes’ rule. Read in moment form, KLA’s posterior-mean update $\mu_t = \eta_t \odot \lambda_t$ is an *uncertainty-gated RNN* (Corollary 3): an input/forget-gated state recurrence whose gates are *precision ratios*. Its

distinguishing feature is the *source* of the gates: unlike Mamba/GLA, or even Gated DeltaNet (GDN), whose gates are *ad-hoc*, input-dependent parameterisations, KLA’s forget and input gates *emerge from Bayes’ rule* - they are supplied by the Möbius precision recursion of Theorem 1, an adaptive, uncertainty-driven learning rate. Table 2 situates KLA among representative linear-attention, gated-RNN, and delta-rule mixers (Yang et al., 2023; 2024; von Oswald et al., 2025; Hatamizadeh et al., 2026) by their gated state updates (extended in Section E), a family also unified through a test-time online-regression lens (Liu et al., 2025; Wang et al., 2025).

5. KLA as a Drop-in Probabilistic Sequence Mixer

We instantiate the parallel Bayesian filter as a sequence-mixing layer with a learnable, Mamba-style block architecture (Section 5.1) and a parameter-efficient state expansion that gives it the associative-recall capacity of a matrix memory (Section 5.2).

5.1. Architecture and parametrisation

To instantiate the above parallel Bayesian filter as a neural network layer, we parametrise the observation value and precision ($\mathbf{v}_t, \Lambda_t^\vee$) and the observation and readout operators ($\mathbf{k}_t, \mathbf{q}_t$) in terms of the layer input sequence $x_{1:T}$, with the observation variance parametrised to be positive. For the continuous stochastic prior (Section 4.1), we treat $\mathbf{a}_t$ and $\mathbf{p}_t$ as learnable, time-invariant parameters $\mathbf{a}$ and $\mathbf{p}$, unlike Mamba, where $\mathbf{a}_t$ is token-dependent and time-varying.

The block follows a fused MLP architecture (Figure 3a). Following Mamba, we pair the KLA sequence mixer with standard scaffolding: a 1D causal convolution (kernel size 4) with SiLU activation, residual connections, and simple nonlinearities; see Figure 5 and Appendix B for further details. We implement efficient Triton (Tillet et al., 2019) and CUDA kernels for the Linear and Möbius parallel scan primitives, but do not fuse the whole block into a single kernel, which other SSMs have shown to be necessary to minimise memory movement and achieve peak performance, especially for the backward pass. The KLA posterior mean recurrence also unrolls into an equivalent matrix multiplication with a lower-triangular “attention” matrix, visualised in Figure 10 (Appendix H).

5.2. Parameter-efficient state expansion

A single scalar latent per channel is too small for associative recall. Given a value $\mathbf{v}_t \in \mathbb{R}^D$, a recall memory calls for a state of dimension $\mathbb{R}^{N \times D}$ (expansion factor N). Projecting keys and values up to ND dimensions would cost $\mathcal{O}(ND d_{\text{model}})$ parameters and tends to overfit; instead, following S4 (Gu et al., 2021) and Mamba (Gu & Dao, 2023),

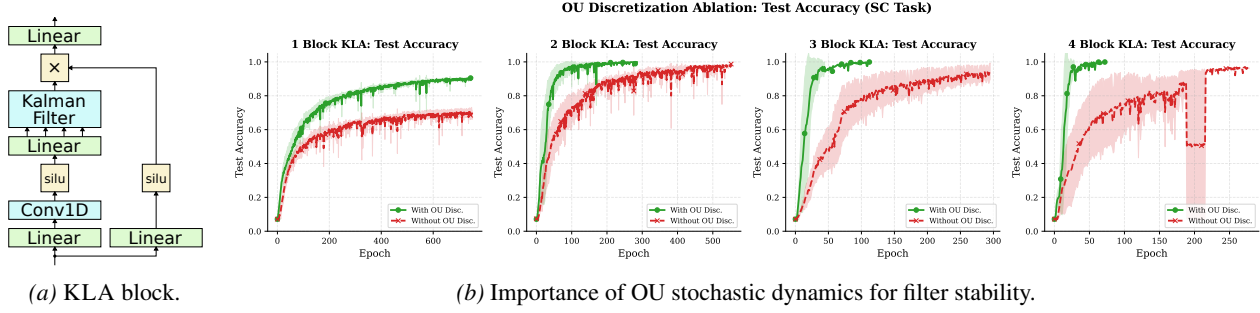


Figure 3. (a) The KLA block follows Mamba’s fused-MLP design, with the Kalman filter as a drop-in mixer (full architecture in Section B). (b) Ablating the OU prior dynamics and discretisation (Section 4.1): OU discretisation improves accuracy and learning stability, especially for deeper models.

Method	Gated state update	Gates from
Linear Attn.	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	-
Mamba-1 (S6)	$\mathbf{S}_t = \bar{\mathbf{A}}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	$\bar{\mathbf{A}}_t$ (ad-hoc)
GDN	$\mathbf{S}_t = \alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$	α_t, β_t (ad-hoc)
KLA (Ours)	$\mathbf{S}_t = \bar{\mathbf{A}} \odot \left(\mathbf{1}\mathbf{1}^\top - \frac{\mathbf{k}_t^2 (\Lambda_t^v)^\top}{\Lambda_t} \right) \odot \mathbf{S}_{t-1} + \frac{\mathbf{k}_t (\Lambda_t^v \mathbf{v}_t)^\top}{\Lambda_t}$	$\bar{\mathbf{A}}, \Lambda_t^v$ Möbius-structured posterior Λ_t

Table 2. **KLA as a gated RNN among sequence mixers.** Each row is a gated state recurrence $\mathbf{S}_t = (\text{forget}) \mathbf{S}_{t-1} + (\text{input}) \mathbf{k}_t \mathbf{v}_t^\top$: orange/green mark the forget/input gates. In Mamba/GLA and the delta-rule variants these gates are *free*, ad-hoc functions of the token; in KLA (blue) they *emerge from Bayes’ rule*, computed from the running precision Λ_t via the Möbius recursion.

each value coordinate $v_t^{(j)}$ - with its precision $\Lambda_t^{v,(j)}$ - is modelled as independently generated by its own N -dimensional latent through a *shared* key $\mathbf{k}_t \in \mathbb{R}^N$. The state $\mathbf{S}_t \in \mathbb{R}^{N \times D}$ is then driven by the outer product $\mathbf{k}_t \mathbf{v}_t^\top$, so the input is projected only to $\mathbf{k}_t \in \mathbb{R}^N$ and to $\mathbf{v}_t, \Lambda_t^v \in \mathbb{R}^D$: capacity grows as the *product* ND while the projections cost only $\mathcal{O}((N+D)d_{\text{model}})$, scaling *additively*. Each of the ND entries then runs an independent scalar filter (a diagonal, mean-field posterior), and the per-channel recursions of Theorems 1 and 2 lift elementwise to their $N \times D$ matrix form: the scalars $(\eta_t, \lambda_t, \mu_t)$ become arrays $(\mathbf{H}_t, \Lambda_t, \mathbf{S}_t)$ related the same way, $\mathbf{S}_t = \mathbf{H}_t \odot \Lambda_t$, with the parallel scan run on the information-form state $\mathbf{H}_t$ and the normalised readout on the moment-form mean $\mathbf{S}_t$, exactly as in the scalar readout (Algorithm 1; Equations (22) and (23), Appendix B). The OU parameters $\bar{\mathbf{A}}, \bar{\mathbf{P}}$ are per-entry but *token-independent*, adding only $\mathcal{O}(ND)$ parameters (no factor of d_{model}), so the expansion remains parameter-efficient even with untied dynamics.

5.3. Training Loss

We use two training losses. (i) **Posterior mean**: logits $\ell_t = g_\theta(\mathbf{y}_t)$ are read out from the posterior mean $\mathbf{y}_t = \mathbf{q}_t \odot \mu_t$ and trained with cross-entropy. (ii) **Log marginal likelihood**: we minimise the negative log marginal likelihood of the output token under the latent posterior (a Monte-Carlo objective), which provides an additional training signal for the posterior variances; the equations and details are given in Appendix B.

6. Experiments

We evaluate KLA as a *drop-in sequence mixer* for language modeling. Our goal is pragmatic: to test whether a Bayesian filtering view of attention can retain the efficiency and accuracy of modern deterministic mixers while yielding interpretable *uncertainty-driven gating*, and, crucially, whether such a probabilistic primitive can be *stacked* into deep networks and trained at scale.

Experimental questions. We address four questions:

- **(Q1) Compute and scaling**: how does the parallel implementation of KLA scale with sequence length, and how does it compare to a recurrent (time-stepped) Kalman implementation in wall-clock and memory?
- **(Q2) Model quality as a single-block primitive**: as a drop-in block, does KLA match or improve on deterministic sub-quadratic alternatives (SSMs / linear attention) on synthetic tasks probing core LM skills (long-range dependence, retrieval, compression)?
- **(Q3) State-tracking expressivity**: does information-form filtering’s fractional-linear structure yield stronger state-tracking (e.g., permutation composition) than linear SSM/attention baselines, while staying scan-parallelisable?
- **(Q4) Stacking and scale**: can KLA be stacked into deep architectures and pretrained at billion-token scale, matching attention and SSM/GLA backbones as a *stand-alone* mixer and *complementing* attention as a hybrid on commonsense benchmarks?

Finally, we ablate architectural choices within the KLA block that enable stable deep stacking of a probabilistic primitive.

6.1. Baselines

We compare KLA against softmax attention and four state-of-the-art sub-quadratic mixers: **GPT** (Vaswani et al., 2017) (softmax attention, $\mathcal{O}(T^2)$ reference); **mLSTM** (Beck et al., 2024) (a modern LSTM with multiplicative interactions and exponential gating); **Mamba** (Gu & Dao, 2023) (a selective SSM with input-dependent transitions); **GDN** (Gated DeltaNet) (Yang et al., 2024), a delta-rule SSM that is the current state of the art among SSM LMs; and **GLA** (Gated Linear Attention) (Yang et al., 2023), a linear-attention variant unifying several linear RNN/SSM formulations. Models are matched on parameter count and effective state size; full configurations are in Appendix J.

6.2. Compute Scaling of Parallel KLA

To make the parallel formulation concrete, we benchmark three KLA implementations that progressively incorporate (i) a naive recurrent Kalman update, (ii) a mathematically parallel formulation via a Torch associative scan, and (iii) hardware-oriented custom Triton and CUDA associative-scan kernels. The scan-based implementations scale efficiently with sequence length compared to the recurrent counterpart, confirming that *information-form filtering admits the same scan-parallel $\mathcal{O}(\log T)$ profile as modern SSM/GLA-style mixers*. The forward+backward (training) runtime curve is shown in Figure 4a; the forward-only (prompt-processing) curve and full setup are in Appendix H.3.

6.3. KLA as a Single-Block Primitive on Synthetic Language Modeling

MAD synthetic LM suite. We evaluate KLA and the baselines as a single-block primitive on the MAD-Lab benchmark (Poli et al., 2024), six synthetic token-manipulation tasks probing different LM skills (Table 8), matching parameter counts and effective state sizes under a common encoder-decoder protocol (Appendices I and J). Evaluating as a single block isolates the *update mechanism itself*, free of depth or memory-budget confounds.

Results. Figure 4b shows KLA matches or surpasses strong deterministic baselines across all tasks. Gains are most pronounced on tasks requiring *selection under corruption* (e.g., *Noisy Recall*, *Fuzzy Recall*, *Selective Copy*), consistent with KLA’s uncertainty-weighted filtering. Probabilistic decoding (KLA+, marginalisation under the learned posterior) improves further on noise-corrupted tasks, suggesting learned posterior variances provide a useful training signal even without an explicit calibration objective.

Posterior uncertainty visualisations. We visualise KLA’s posterior variance on Selective Copy (Figure 4c) as a diagnostic of its filtering dynamics, not a claim of calibrated uncertainty: variance generally decreases as evidence accumulates, while sharp spikes align with copy-relevant positions.

Additional experiments on long-context associative recall (MQAR) (Arora et al., 2023), where KLA outperforms Mamba and GLA, are reported in Appendix H.2.

6.4. State Tracking and Permutation-Group Expressivity

Why A_5 matters. A central open question is whether sub-quadratic sequence models can *track state* beyond highly-parallel shortcut computation (Merrill & Sabharwal, 2023; Merrill et al., 2024). Recent theory shows common linear/diagonal SSMs (e.g. S4, Mamba), despite their recurrent form, share the bounded-depth limits of transformers: simulable by highly-parallel circuit classes (TC^0), they cannot solve inherently sequential state-tracking whose difficulty grows with sequence length. The word problem for the alternating group A_5 (the smallest non-solvable subgroup of S_5) is NC^1 -complete and a minimal benchmark for *hard state tracking* (Merrill et al., 2024).

Result. Following Merrill et al. (2024), Figure 1(a) shows KLA solves A_5 at constant depth (1-2 layers), whereas linear SSM/attention baselines need depth growing with sequence length. Among the parallel-trainable architectures we consider, it is the only one to do so; concurrent work attains state tracking in linear RNNs via negative eigenvalues (Grazzi et al., 2025), whereas KLA achieves it through the Möbius recurrence’s nonlinearity. Its *fractional-linear* (information-form) updates evolve the state more richly than a linear recurrence, placing KLA beyond TC^0 while remaining scan-parallelisable.

6.5. Stacking KLA for Language Modeling at Scale

The experiments above isolate KLA as a single (1-2 block) primitive. We now ask whether it *stacks into deep networks and trains at scale*: we pretrain KLA and all baselines from scratch on OpenFineWeb-Edu at **45M** (1.8B tokens) and **180M** (10.9B tokens) parameters, matched on optimiser and token budget, reporting zero-shot accuracy on eight common-sense benchmarks (protocol in Appendix J). To our knowledge this is the first large-scale from-scratch pretraining of a stacked probabilistic-filtering primitive. Stacking also raises an interface question absent for deterministic mixers: KLA emits a full posterior per step; as a first iteration we propagate only the *mean* between blocks, leaving full-posterior propagation to future work.

Standalone KLA is competitive. Table 3 shows KLA is competitive with softmax attention, Mamba, and GDN at both scales (with per-task strengths on HellaSwag and PIQA).

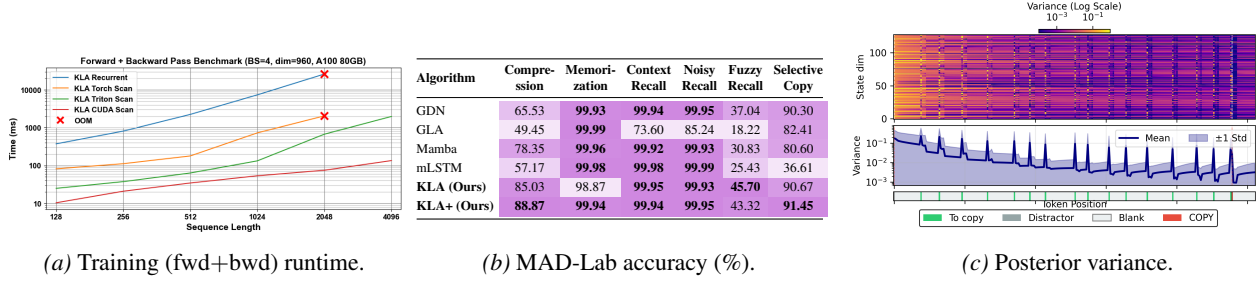


Figure 4. KLA as a single-block primitive: efficiency, quality, and uncertainty. (a) Forward+backward (training) runtime vs. sequence length; scan-based KLA matches the $\mathcal{O}(\log T)$ profile of modern mixers (forward-only curve in Section H.3). (b) MAD-Lab synthetic-LM accuracy (GDN, GLA, Mamba, mLSTM, KLA, and probabilistic decoding KLA+); all methods are parameter- and state-matched; darker shading = higher accuracy. (c) Posterior variance on Selective Copy: spikes align with copy-relevant tokens, an interpretable filtering signal.

Model	LAMB.	HellaS. ⁿ	PIQA	Arc-E	Arc-C ⁿ	WinoGr.	OBQA	BoolQ	Average
GPT-45M	16.00	28.60	58.00	40.20	20.60	50.00	27.40	37.40	34.78
Mamba-45M	9.20	29.20	59.80	37.20	25.40	48.80	25.40	61.60	37.08
GDN-45M	16.00	28.20	58.40	39.00	<u>23.40</u>	52.60	<u>27.20</u>	45.20	36.25
KLA-45M	7.60	28.80	58.60	40.60	23.20	<u>52.20</u>	27.40	63.00	<u>37.68</u>
GPT+KLA-45M	16.00	28.20	<u>58.60</u>	42.80	23.00	<u>50.80</u>	24.40	<u>62.80</u>	38.33
GPT-180M	28.80	32.20	63.40	51.00	27.00	49.20	31.20	54.80	42.20
Mamba-180M	24.00	30.60	63.00	51.00	27.20	53.80	33.00	55.80	42.30
GDN-180M	27.80	31.60	60.40	45.60	25.80	53.40	30.00	<u>60.80</u>	41.93
KLA-180M	22.40	<u>34.00</u>	63.00	50.00	26.00	<u>53.60</u>	<u>32.40</u>	<u>60.80</u>	<u>42.78</u>
GPT+KLA-180M	30.20	34.40	64.20	55.80	29.00	51.80	31.00	63.20	44.95

Table 3. Language modeling at two academic scales. Zero-shot accuracy (% , $\uparrow$) on eight commonsense benchmarks at 45M (1.8B tokens) and 180M (10.9B tokens); ⁿ denotes length-normalised accuracy (acc_n). **Bold** = best within each size block, underline = second. Rows above the dashed line are standalone mixers; the row below it is the hybrid GPT+KLA, in which only the *final* attention layer of GPT is replaced with a KLA block.

Its advantage is largest at shallow depth and narrows as baselines add layers, as predicted by our expressivity analysis (Section 6.4). It underperforms on the *clean* copying task LAMBADA, consistent with uncertainty-weighted updates favouring robust over verbatim recall (cf. Section 6.3).

A single KLA layer improves a GPT. Unlike prior hybrids that interleave many layers, we replace only the *final* layer of a softmax-attention GPT with an SSM-like block (KLA, Mamba, or GDN), motivated by evidence that uncertainty concentrates in later layers (Agarwal et al., 2025). At 180M, GPT+KLA improves over pure GPT and over the analogous GPT+Mamba and GPT+GDN hybrids on average and most tasks (Table 3; Figure 1(b)), showing KLA is an effective drop-in *complement* to attention, not merely a substitute.

6.6. Ablations

Finally, we isolate the two modeling choices that matter most. Stable *deep stacking* of a probabilistic primitive was possible *only* because of the mean-reverting Ornstein-Uhlenbeck (OU) prior (Section 4.1): naive stacking was unstable and standard fixes did not help, whereas the OU prior keeps the latent dynamics bounded, letting KLA stack to 12 and 24

blocks stably (Figure 3b confirms improved stability and accuracy, especially at depth). A second ablation (Figure 7, Appendix H.1) shows the process-noise parameter supplies the nonlinear gating: setting $\mathbf{p}_t=0$ collapses the Möbius precision recurrence to a fixed-gate linear recurrence and degrades the selection-heavy MAD tasks (Selective Copy –14.9, Compression –12.1; 4.7 on average).

7. Conclusion

In this paper, we propose a scalable probabilistic sequence-modeling primitive, the KLA layer, which treats language modeling as a state estimation problem from noisy observations. We show that posterior inference in these models is tractable for very long sequences via parallel scans, and that the resulting gating mechanisms are strictly more expressive than those of modern deterministic linear SSMs and RNNs. Our experiments indicate the promise of KLA as a scalable primitive within the language and probabilistic sequence-modeling toolbox (**Limitations** and future work in Appendix A).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we believe must be specifically highlighted here. The KLA layer provides principled uncertainty quantification, which could benefit applications requiring calibrated predictions, though, as with any language modeling primitive, deployment should consider potential misuse.

Acknowledgements

We thank Henry Gouk, Nikolay Malkin, Thomas Lee, and Stephan Kostov for their valuable feedback on this draft at various stages. We also thank Wouter Boomsma, Frederikke Isa Marin, Jose Cano Reyes, and Jude Harris for the informative discussions over the course of this research. This project has received funding from the European Union’s Horizon Europe research and innovation programme under grant agreement No. 101120726. This work was funded by UK Research and Innovation (UKRI) under the UK government’s Horizon Europe funding Guarantee 10085198. The authors acknowledge the use of resources provided by the Isambard-AI National AI Research Resource (AIRR) (McIntosh-Smith et al., 2024). Isambard-AI is operated by the University of Bristol and is funded by the UK Government’s Department for Science, Innovation and Technology (DSIT) via UK Research and Innovation; and the Science and Technology Facilities Council [ST/AIRR/I-A-I/1023].

References

- Agarwal, N., Dalal, S. R., and Misra, V. The bayesian geometry of transformer attention. *arXiv preprint arXiv:2512.22471*, 2025.
- Alonso, C. A., Sieber, J., and Zeilinger, M. N. State space models as foundation models: A control theoretic overview. In *2025 American Control Conference (ACC)*, pp. 146–153. IEEE, 2025.
- Anderson, B. D. and Moore, J. B. *Optimal filtering*. Courier Corporation, 2005.
- Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., and Ré, C. Zoology: Measuring and improving recall in efficient language models. *arXiv preprint arXiv:2312.04927*, 2023.
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. xLSTM: Extended long short-term memory. *Advances in Neural Information Processing Systems*, 37: 107547–107603, 2024.
- Becker, P., Pandya, H., Gebhardt, G., Zhao, C., Taylor, C. J., and Neumann, G. Recurrent Kalman networks: Factorized inference in high-dimensional deep feature spaces. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 544–552. PMLR, 2019.
- Becker, P., Freymuth, N., and Neumann, G. Kalmamba: Towards efficient probabilistic state space models for rl under uncertainty. In *ICML 2024 Workshop: Aligning Reinforcement Learning Experimentalists and Theorists*, 2024.
- Blelloch, G. E. Prefix sums and their applications. In *Synthesis of Parallel Algorithms*, pp. 35–60. Morgan Kaufmann, 1990.
- Broadbent, D. E. A mechanical model for human attention and immediate memory. *Psychological review*, 64(3):205, 1957.
- Dao, T. and Gu, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
- Deutsch, J. A. and Deutsch, D. Attention: Some theoretical considerations. *Psychological review*, 70(1):80, 1963.
- Feldman, H. and Friston, K. J. Attention, uncertainty, and free-energy. *Frontiers in human neuroscience*, 4:215, 2010.
- Fraccaro, M., Sønderby, S. K., Paquet, U., and Winther, O. Sequential neural models with stochastic layers. In *Advances in Neural Information Processing Systems*, volume 29, pp. 2207–2215, 2016.
- Grazzi, R., Siems, J., Franke, J. K. H., Zela, A., Hutter, F., and Pontil, M. Unlocking state-tracking in linear RNNs through negative eigenvalues. In *International Conference on Learning Representations (ICLR)*, pp. 36565–36597, 2025.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Gu, A., Goel, K., and Re, C. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2021.
- Gupta, A., Gu, A., and Berant, J. Diagonal state spaces are as effective as structured state spaces. In *Advances in Neural Information Processing Systems*, volume 35, pp. 22982–22994, 2022.
- Haarnoja, T., Tang, H., Abbeel, P., and Levine, S. Reinforcement learning with deep energy-based policies. In *International Conference on Machine Learning*, volume 70

- of *Proceedings of Machine Learning Research*, pp. 1352–1361. PMLR, 2017.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 2555–2565. PMLR, 2019.
- Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- Hatamizadeh, A., Choi, Y., and Kautz, J. Gated deltanet-2: Decoupling erase and write in linear attention. *arXiv preprint arXiv:2605.22791*, 2026.
- Kalman, R. E. A new approach to linear filtering and prediction problems. *Transactions of the ASME—Journal of Basic Engineering*, 82(Series D):35–45, 1960.
- Karl, M., Sölch, M., Bayer, J., and van der Smagt, P. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. In *Proceedings of the International Conference on Learning Representations*, 2017.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, pp. 5156–5165. PMLR, 2020.
- Khan, M. E. Matrix inversion lemma and information filter. Technical report, Honeywell Technology Solutions Lab, Bangalore, India, 2005.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Krishnan, R. G., Shalit, U., and Sontag, D. Deep Kalman filters. *arXiv preprint arXiv:1511.05121*, 2015.
- Krishnan, R. G., Shalit, U., and Sontag, D. Structured inference networks for nonlinear state space models. In *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, pp. 2101–2109, 2017.
- Liu, B., Wang, R., Wu, L., Feng, Y., Stone, P., and Liu, Q. Longhorn: State space models are amortized online learners. In *International Conference on Learning Representations*, volume 2025, pp. 95419–95434, 2025.
- Luis, C. E., Bottero, A. G., Vinogradskaya, J., Berkenkamp, F., and Peters, J. Uncertainty representations in state-space layers for deep reinforcement learning under partial observability. *arXiv preprint arXiv:2409.16824*, 2024.
- McIntosh-Smith, S., Alam, S., and Woods, C. Isambard-AI: a leadership-class supercomputer optimised specifically for Artificial Intelligence. In *Proceedings of the Cray User Group*, pp. 44–54. 2024.
- Merrill, W. and Sabharwal, A. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.
- Merrill, W., Petty, J., and Sabharwal, A. The illusion of state in state-space models. *arXiv preprint arXiv:2404.08819*, 2024.
- Muca Cirone, N., Orvieto, A., Walker, B., Salvi, C., and Lyons, T. Theoretical foundations of deep selective state-space models. *Advances in Neural Information Processing Systems*, 37:127226–127272, 2024.
- Nielsen, F. and Garcia, V. Statistical exponential families: A digest with flash cards. *arXiv preprint arXiv:0911.4863*, 2009.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- Penedo, G., Kydlíček, H., allal, L. B., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., and Wolf, T. The fineweb datasets: Decanting the web for the finest text data at scale. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 30811–30849, 2024.
- Poli, M., Thomas, A. W., Nguyen, E., Ponnusamy, P., Deiseroth, B., Kersting, K., Suzuki, T., Hie, B., Ermon, S., Re, C., Zhang, C., and Massaroli, S. Mechanistic design and scaling of hybrid architectures. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 40908–40950. PMLR, 2024.
- Rao, R. P. and Ballard, D. H. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature neuroscience*, 2(1):79–87, 1999.
- Särkkä, S. and García-Fernández, Á. F. Temporal parallelization of bayesian smoothers. *IEEE Transactions on Automatic Control*, 66(1):299–306, 2021.

- Shaj, V., Becker, P., Büchler, D., Pandya, H., van Duijken, N., Taylor, C. J., Hanheide, M., and Neumann, G. Action-conditional recurrent kalman networks for forward and inverse dynamics learning. In *Conference on Robot Learning*, pp. 765–781. PMLR, 2021.
- Shaj, V., Gholam Zadeh, S., Demir, O., Douat, L., and Neumann, G. Multi time scale world models. *Advances in Neural Information Processing Systems*, 36, 2023.
- Smith, J. T., Warrington, A., and Linderman, S. Simplified state space layers for sequence modeling. In *The Eleventh International Conference on Learning Representations*, 2022.
- Storkey, A. and Valabregue, R. Hopfield learning rule with high capacity storage of time-correlated patterns. *Electronics Letters*, 33:1803 – 1804, 11 1997.
- Team, K., Zhang, Y., Lin, Z., Yao, X., Hu, J., Meng, F., Liu, C., Men, X., Yang, S., Li, Z., et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.
- Tillet, P., Kung, H.-T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 10–19, 2019.
- Treisman, A. M. Strategies and models of selective attention. *Psychological review*, 76(3):282, 1969.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- von Oswald, J., Scherrer, N., Kobayashi, S., Versari, L., Yang, S., Schlegel, M., Maile, K., Schimpf, Y., Sieberling, O., Meulemans, A., et al. Mesanet: Sequence modeling by locally optimal test-time training. *arXiv preprint arXiv:2506.05233*, 2025.
- Wang, K. A., Shi, J., and Fox, E. B. Test-time regression: a unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025.
- Watter, M., Springenberg, J. T., Boedecker, J., and Riedmiller, M. A. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, volume 28, pp. 2746–2754, 2015.
- Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving Mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.
- Yu, A. and Erichson, N. B. Block-biased mamba for long-range sequence processing. *arXiv preprint arXiv:2505.09022*, 2025.

Appendix

Appendix outline

- [Appendix A: Limitations and Future Work \(p. 14\)](#)
- [Appendix B: Architecture Details \(p. 14\)](#)
 - [Figure 5: Model Architecture Diagram \(p. 14\)](#)
 - [Algorithm 1: Kalman Linear Attention \(p. 15\)](#)
 - [Training Loss](#)
- [Appendix C: Notation \(p. 17\)](#)
- [Appendix D: Extended Background \(p. 18\)](#)
 - [Mobius / Fractional-Linear Transformations \(p. 18\)](#)
 - [Information Form of Gaussian Distributions \(p. 18\)](#)
 - [Kalman and Information Filters \(p. 18\)](#)
- [Appendix E: Gated-Update View: KLA among Linear-Attention and Gated-RNN Mixers \(p. 20\)](#)
- [Appendix F: Theorems and Proofs \(p. 21\)](#)
- [Appendix G: Probabilistic Interpretation and State-Expanded Recursions \(p. 25\)](#)
- [Appendix H: Additional Empirical Results \(p. 27\)](#)
 - [Ablation On The Importance Of Process Noise Parameter \(p. 27\)](#)
 - [Forward Pass Runtime Scaling \(p. 28\)](#)
 - [Equivalent Attention Matrix \(p. 28\)](#)
 - [Attention Map Visualisation \(p. 28\)](#)
- [Appendix I: Datasets and Benchmarks Used \(p. 32\)](#)
 - [MAD LM Suite \(p. 32\)](#)
 - [Long-Context MQAR \(p. 33\)](#)
- [Appendix J: Implementation Details \(p. 34\)](#)
 - [Experimental Protocol \(p. 34\)](#)
 - [Training Hyperparameters \(p. 34\)](#)
 - [MAD-Lab Hyperparameters \(p. 34\)](#)
 - [MQAR Hyperparameters \(p. 34\)](#)
 - [A5 State Tracking Hyperparameters \(p. 35\)](#)

A. Limitations and Future Work

This paper is a *first iteration* on a fundamental algorithmic family, Bayesian filters as parallelisable sequence-modeling primitives, and we deliberately start from the simplest possible assumptions: diagonal, linear time-invariant (LTI) dynamics with a linear-Gaussian observation model. Richer yet compute-friendly variants (e.g., structured or input-dependent dynamics, non-diagonal couplings, and nonlinear or learned observation models) are natural next steps that could close the gap to full Kalman filtering while retaining the scan-parallel profile.

We have also not explicitly exploited the posterior covariance for applications where uncertainty quantification may be critical (e.g., epistemic prompt uncertainty, hallucination detection, and out-of-distribution prompt detection), which are likely areas where KLA could excel; we leave this for future work.

Finally, because the underlying state-space formulation is inherently continuous, a promising direction is to move beyond the language domain and train vision and multimodal foundation models over continuous modalities, where explicit belief-state uncertainty may be especially natural.

B. Architecture Details

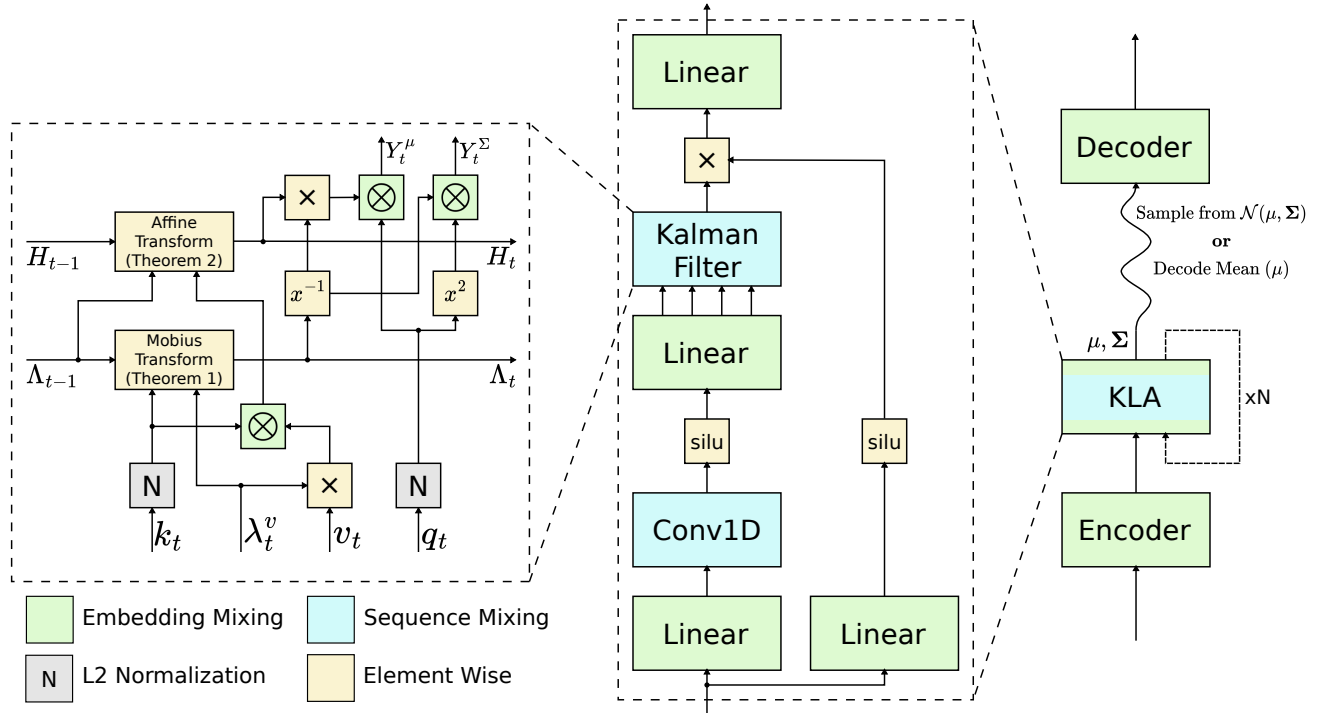


Figure 5. Model architecture used in our experiments. The KLA block is a drop-in replacement for Transformer or SSM blocks. We follow a similar block structure to Mamba, fusing the sequence mixing and gated MLP into a single block. We additionally employ QK-Norm and use an expansion factor of 1. Tokens are embedded and passed through a *sequence mixer*, then decoded to logits. The sequence mixer is a drop-in primitive that can be instantiated as attention variants, or other SSM/Filter modules. Many mixers are paired with standard *scaffolding*-e.g., 1D convolutions, residual/skip connections, and simple nonlinearities (elementwise multiplication or activations)-shown on the right.

Architecture. We use a consistent language modeling architecture across all models and baselines, as illustrated in Figure 5. The architecture follows a standard pattern: input tokens are embedded, passed through a sequence mixer layer, and then decoded to output logits. The sequence mixer serves as a drop-in primitive that can be instantiated as KLA, Mamba, GLA, or other SSM/attention variants.

Following the Mamba design, we employ standard scaffolding components paired with the sequence mixer: a 1D causal convolution with kernel size 4 and SiLU activation, along with residual connections and simple nonlinearities (elementwise multiplication). We use standard state expansion (see Section 3) and is set to 16 (N or sometimes called d_{state} in SSM codebases) as default similar to (Gu & Dao, 2023). More extensive exploration of scaffolding configurations optimally suited to the KLA filtering mixer can be a direction for future work.

Algorithm 1 Kalman Linear Attention

```

1: Input:  $\mathbf{X} : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D})$ 
2: Output  $\mathbf{Y}^\mu : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D})$ 
3: Output (Optional)  $\mathbf{Y}^\Sigma : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D})$ 
4:  $\mathbf{A}, \mathbf{P}, \Delta : (\mathbf{N}, \mathbf{D}) \leftarrow \text{Parameters}$  ▷ A: Gating/decay, P: Process noise, Δ: Discretisation step size
5:  $\mathbf{K} : (\mathbf{B}, \mathbf{T}, \mathbf{N}, 1) \leftarrow f_{\theta_k}(\mathbf{X})$  ▷ Observation operator
6:  $\mathbf{Q} : (\mathbf{B}, \mathbf{T}, 1, \mathbf{N}) \leftarrow f_{\theta_q}(\mathbf{X})$  ▷ Readout operator
7:  $\mathbf{V}, \Lambda^v : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D}) \leftarrow f_{\theta_v, \lambda^v}(\mathbf{X})$  ▷ Observation mean and precision,  $\lambda^v \in \mathbb{R}^+$ 
8:  $\bar{\mathbf{A}} : (\mathbf{N}, \mathbf{D}) \leftarrow \text{OU-Discretisation}(\mathbf{A}, \Delta)$  ▷ See Equation (8)
9:  $\bar{\mathbf{P}} : (\mathbf{N}, \mathbf{D}) \leftarrow \text{OU-Discretisation}(\mathbf{A}, \mathbf{P}, \Delta)$ 
10:  $\Lambda : (\mathbf{B}, \mathbf{T}, \mathbf{N}, \mathbf{D}) \leftarrow \text{Mobius-Scan}(\mathbf{K}, \Lambda^v, \bar{\mathbf{P}}, \bar{\mathbf{A}})$  ▷ matrix form: Table 4
11:  $\mathbf{H} : (\mathbf{B}, \mathbf{T}, \mathbf{N}, \mathbf{D}) \leftarrow \text{Linear-Scan}(\mathbf{K}, \Lambda^v, \mathbf{V}, \Lambda, \bar{\mathbf{P}}, \bar{\mathbf{A}})$  ▷ information mean  $\eta$ ; matrix form: Table 4
12:  $\mathbf{S} : (\mathbf{B}, \mathbf{T}, \mathbf{N}, \mathbf{D}) \leftarrow \mathbf{H} \oslash \Lambda$  ▷ moment-form state ( $\mathbf{S}_t$  of Table 2):  $\mu = \eta \oslash \lambda$ 
13:  $\mathbf{Y}^\mu : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D}) \leftarrow \mathbf{Q}\mathbf{S}$ 
14: if Decode Variance then
15:    $\mathbf{Y}^\Sigma : (\mathbf{B}, \mathbf{T}, 1, \mathbf{D}) \leftarrow \mathbf{Q}^2 \Lambda^{-1}$ 
16: Return:  $\mathbf{Y}^\mu, \mathbf{Y}^\Sigma$ 
17: else
18:   Return:  $\mathbf{Y}^\mu$ 
19: end if
    
```

	Scalar ($N=1$)	Matrix-structured ($N>1$)
Precision	$\lambda_t = \frac{(1 + \bar{\mathbf{p}} \odot \phi_t) \odot \lambda_{t-1} + \bar{\mathbf{a}}^2 \odot \phi_t}{\bar{\mathbf{p}} \odot \lambda_{t-1} + \bar{\mathbf{a}}^2}$	$\Lambda_t = \frac{(1 + \bar{\mathbf{P}} \odot \Phi_t) \odot \Lambda_{t-1} + \bar{\mathbf{A}}^2 \odot \Phi_t}{\bar{\mathbf{P}} \odot \Lambda_{t-1} + \bar{\mathbf{A}}^2}$
Information mean	$\eta_t = \mathbf{f}_t \odot \eta_{t-1} + \mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t,$ $\mathbf{f}_t = \bar{\mathbf{a}} \odot (\bar{\mathbf{a}}^2 + \bar{\mathbf{p}} \odot \lambda_{t-1})$	$\eta_t = \mathbf{F}_t \odot \eta_{t-1} + \mathbf{k}_t (\Lambda_t^v \odot \mathbf{v}_t)^\top,$ $\mathbf{F}_t = \bar{\mathbf{A}} \odot (\bar{\mathbf{A}}^2 + \bar{\mathbf{P}} \odot \Lambda_{t-1})$
Readout	$\mathbf{y}_t = \mathbf{q}_t \odot (\eta_t \oslash \lambda_t)$	$\mathbf{y}_t = \mathbf{S}_t^\top \mathbf{q}_t, \quad \mathbf{S}_t = \eta_t \oslash \Lambda_t$

Table 4. KLA recursions computed by Algorithm 1: scalar ($N=1$) vs. matrix-structured ($N>1$). All operations are elementwise; $\bar{\mathbf{A}}, \bar{\mathbf{P}}$ are the discretised OU decay and process noise, Λ_t^v the value precision, and the precision-weighted squared key is $\phi_t = \mathbf{k}_t^2 \odot \Lambda_t^v$ (scalar) / $\Phi_t = \mathbf{k}_t^2 (\Lambda_t^v)^\top$ (matrix). Setting $N=1$ collapses the outer products to elementwise products, recovering the scalar per-channel recursions. Discussed in Section 5.2 and Section G.2.

We implemented KLA with PyTorch (Paszke et al., 2019) and used the Adam optimiser (Kingma & Ba, 2015) for training the models.

Training loss. We consider two training losses. Throughout, the decoder is applied to the query-conditioned readout $\mathbf{y}_t = \mathbf{q}_t \odot \mathbf{z}_t$; when the readout noise is zero this is a deterministic linear map, so conditioning on $\mathbf{z}_t$ or $\mathbf{y}_t$ is equivalent up to this transformation.

(i) **Posterior mean:** logits $\ell_t = g_\theta(\mathbf{y}_t)$ are obtained from the posterior-mean readout $\mathbf{y}_t = \mathbf{q}_t \odot \mu_t$ and trained with cross-entropy.

(ii) **Log marginal likelihood:** minimises the negative log marginal likelihood of the output token o_t under the latent posterior,

$$\begin{aligned}
 -\log p_\theta(o_t | \mathbf{X}) &= -\log \int \underbrace{p_\theta(o_t | \mathbf{y}_t)}_{\text{decoder}} \underbrace{p_\theta(\mathbf{z}_t | \mathbf{X})}_{\text{KLA posterior}} d\mathbf{z}_t \\
 &\approx -\log \left(\frac{1}{S} \sum_{s=1}^S p_\theta(o_t | \mathbf{y}_t^{(s)}) \right), \tag{18}
 \end{aligned}$$

with $\mathbf{z}_t^{(s)} \sim p_\theta(\mathbf{z}_t \mid \mathbf{X})$ and $\mathbf{y}_t^{(s)} = \mathbf{q}_t \odot \mathbf{z}_t^{(s)}$. Equivalently, the estimator can be written as a logsumexp:

$$\begin{aligned}
 & -\log p_\theta(o_t \mid \mathbf{X}) \\
 & \approx -\log \left(\frac{1}{S} \sum_{s=1}^S p_\theta(o_t \mid \mathbf{y}_t^{(s)}) \right) \\
 & = -\text{logsumexp}_{s=1}^S \left(\log p_\theta(o_t \mid \mathbf{y}_t^{(s)}) \right) + \log S,
 \end{aligned} \tag{19}$$

which reduces to cross-entropy when $S=1$. We show this for a single time-step; it can be trivially extended across timesteps.

Attention	Deterministic SSM / GLA	KLA (probabilistic)
Query operator $\mathbf{q}_t$	Output/readout map $\mathbf{C}_t$	Query/readout operator $\mathbf{q}_t$
Key operator $\mathbf{k}_t$	Input to latent map $\mathbf{b}_t$	Observation model/operator $\mathbf{k}_t$
Value $\mathbf{v}_t$	Token input/control $\mathbf{u}_t$	Noisy observation $\mathbf{v}_t$
Output $\mathbf{y}_t$	Output $\mathbf{y}_t$	Output $\mathbf{y}_t$
<i>Hidden state (implicit in attention, explicit in SSMs/KLA):</i>		
-	Hidden state $\mathbf{h}_t$	Posterior belief $\mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Lambda}_t^{-1})$ over $\mathbf{z}_t$
-	State decay $\bar{\mathbf{a}}_t = e^{-\mathbf{a}(t)\Delta t}$	State decay $\bar{\mathbf{a}}_t = e^{-\mathbf{a}\Delta t}$
<i>Uncertainty terms (explicit in KLA only):</i>		
-	-	Value precision $\boldsymbol{\Lambda}_t^v$
-	-	State precision $\boldsymbol{\lambda}_t$
-	-	Process noise $\bar{\mathbf{p}}_t = \frac{\mathbf{p}_t^2}{2\mathbf{a}} \odot (1 - e^{-2\mathbf{a}\Delta t})$

Table 5. Notation alignment across attention, SSMs, and KLA. Precisions ($\boldsymbol{\lambda}$, $\boldsymbol{\Lambda}^v$, $\boldsymbol{\Lambda}^p$) are inverse variances encoding confidence: higher precision means lower uncertainty.

C. Notation

Throughout the paper we use the following convention:

Symbol	Meaning
t	Time index
$\mathbf{z}_t$	Latent state at time t
$\mathbf{v}_t$	Value (observed token/features) at time t
$\mathbf{A}_t$	State transition matrix
$\mathbf{P}_t$	Process noise covariance (discretized: $\bar{\mathbf{P}}_t$)
$\mathbf{C}_t$	Observation (likelihood) matrix
$\boldsymbol{\Lambda}_t^v$	Value precision (inverse observation variance)
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	Gaussian in <i>moment</i> form
$\mathcal{N}(\boldsymbol{\eta}, \boldsymbol{\Lambda})$	Gaussian in <i>canonical</i> form; $\boldsymbol{\eta} = \boldsymbol{\Lambda}\boldsymbol{\mu}$, $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$
$\boldsymbol{\mu}_t^{\text{prior}}, \boldsymbol{\Sigma}_t^{\text{prior}}$	Predictive prior mean/cov at t (canonical: $\boldsymbol{\eta}_t^{\text{prior}}, \boldsymbol{\Lambda}_t^{\text{prior}}$)
$\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t$	Posterior mean/cov at t (canonical: $\boldsymbol{\eta}_t, \boldsymbol{\Lambda}_t$)
Beliefs (linear-Gaussian case):	
$\mathbf{z}_t \mid \mathbf{v}_{1:t-1} \sim \mathcal{N}(\boldsymbol{\mu}_t^{\text{prior}}, \boldsymbol{\Sigma}_t^{\text{prior}})$	(predictive prior)
$\mathbf{z}_t \mid \mathbf{v}_{1:t} \sim \mathcal{N}(\boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)$	(posterior)

Diagonal (per-dimension) specialisation. When matrices are diagonal, we identify them with their diagonal vectors and switch to bold lowercase: $\mathbf{A}_t \equiv \mathbf{a}_t$, $\mathbf{B}_t \equiv \mathbf{b}_t$, $\mathbf{K}_t \equiv \mathbf{k}_t$ (key), $\mathbf{P}_t \equiv \mathbf{p}_t$ (process noise); value precision $\boldsymbol{\Lambda}_t^v$ is treated as a scalar. All algebra becomes *elementwise*: products use the Hadamard operator $\odot$, superscripts like $\mathbf{a}_t^2$ denote elementwise squares, and $(\cdot)^{-1}$ on lowercase vectors means elementwise inverse.

Full (matrix)	Diagonal notation (this paper)	Implication (elementwise)
$\mathbf{A}_t$	$\mathbf{a}_t$	$(\mathbf{A}_t \mathbf{x}) = \mathbf{a}_t \odot \mathbf{x}$; $\mathbf{A}_t^2 \Rightarrow \mathbf{a}_t^2$
$\mathbf{K}_t$	$\mathbf{k}_t$	$\mathbf{K}_t \boldsymbol{\Sigma} \mathbf{K}_t^\top \Rightarrow \mathbf{k}_t^2 \odot \boldsymbol{\Sigma}$ (key)
$\mathbf{P}_t$	$\mathbf{p}_t$	Process noise cov.; $\bar{\mathbf{P}}_t \Rightarrow \bar{\mathbf{p}}_t$ (discretized)
$\boldsymbol{\Lambda}_t^v$	$\boldsymbol{\Lambda}_t^v$	Scalar value precision (variance = $(\boldsymbol{\Lambda}_t^v)^{-1}$)

D. Extended Background

D.1. Möbius / Fractional-Linear Transformations.

A Möbius (a.k.a. fractional-linear) transformation is the map

$$z \mapsto \frac{az + b}{cz + d}, \quad ad - bc \neq 0.$$

It generalises the familiar affine form ($c = 0$ gives $az + b$) by introducing an additional denominator term. Each transform is represented (up to multiplication by a nonzero scalar) by a 2×2 matrix $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ and acts on a scalar z via the linear-fractional rule

$$M(z) := \frac{az + b}{cz + d}.$$

Composing multiple transforms amounts to multiplying their representing matrices, so associativity and invertibility (whenever $\det M \neq 0$) follow directly. Geometrically, Möbius transforms can be viewed as compositions of translations, rotations/scalings, and a crucial nonlinear ingredient—*inversion*—which affine transformations cannot capture. The denominator also provides a built-in *self-normalisation*: as the state grows, it is automatically rescaled, producing saturation and stability effects reminiscent of gating or normalisation layers, but achieved directly through the update itself.

D.2. Information Form of Gaussian Distributions and Information Filters.

Gaussians belong to the exponential family (Nielsen & Garcia, 2009), so the posterior can be written in *canonical* (information) form with precision $\Lambda_t = \Sigma_t^{-1}$ and natural parameter $\eta_t = \Lambda_t \mu_t$. Filtering recursions in this parameterisation are known as *information filters (IF)* (Anderson & Moore, 2005; Khan, 2005). They are algebraically equivalent to the standard (moment) Kalman updates; only the representation of the Gaussian belief differs. Crucially, the canonical posterior updates are *linear-additive* in (η, Λ) , a property we leverage for scalable sequence modeling in Section 4.

D.3. Control vs Observation View Of Token Processing

State-space models can be understood from two complementary perspectives (Figure 6):

Observation view (traditional SSM/Kalman filter). In classical Bayesian filtering, input tokens are treated as *noisy observations* of a hidden state. The model assumes: (i) a latent state z_t evolves over time via learned dynamics, and (ii) each token v_t is a noisy measurement generated from this state. The goal is to *infer* the hidden state from the observed tokens—asking “what underlying state could have produced these observations?”

Control view (Mamba/S6). In contrast, Mamba treats input tokens as *control signals* that directly drive the state. There is no observation noise or inference step; tokens deterministically update the hidden state via $z_t = A_t z_{t-1} + B_t u_t$. The state simply accumulates information from inputs—asking “how should I update my memory given this new input?”

Why this matters. The observation view naturally provides uncertainty quantification: since tokens are noisy measurements, the model maintains a posterior *distribution* over states rather than a point estimate. This is the foundation of KLA—we treat language modeling as Bayesian filtering where tokens inform our belief about an underlying semantic state.

D.4. Kalman and Information Filters

D.4.1. BAYESIAN FILTERING AS POSTERIOR INFERENCE.

Bayesian filters (Kalman, information-filter variants) are *inference schemes*—procedures that compute posterior beliefs about latent states from noisy data. The Kalman filter assumes a Gaussian linear state-space model: the latent evolves via a (possibly time-varying) linear transition A_t with unmodelled variability captured by process noise Q_t , and each observed token feature o_t is generated by an observation model C_t with observation noise covariance Σ_t^{obs} . The filter then computes $p(z_t \mid o_{1:t})$ sequentially by combining the dynamics prior with token evidence.

In this linear-Gaussian setting, inference has closed-form solutions: in the *moment parametrisation* (μ_t, Σ_t) the scheme is called the **Kalman filter**, while in the *canonical parametrisation* (η_t, Λ_t) it is called the **information filter**. Both are algebraically equivalent, differing only in how Gaussian beliefs are represented.

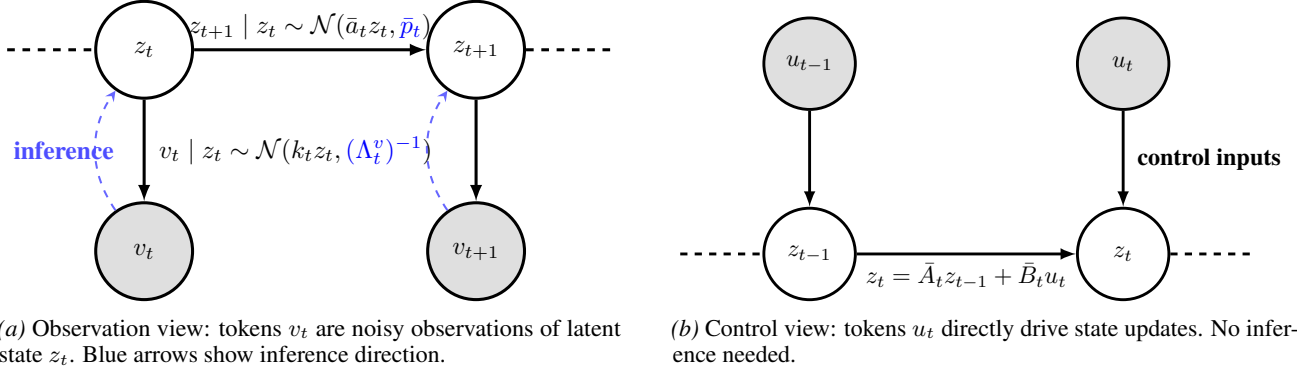


Figure 6. Two views of state-space sequence models. **Left:** Bayesian/observation view-tokens are measurements, inference recovers hidden state. **Right:** Control view (Mamba)-tokens are inputs that deterministically update state.

Belief / Parameters	Kalman Filter (moment form)	Information Filter (canonical form)
Prior / Prediction $z_t v_{1:t-1} \sim \mathcal{N}(\mu_t^{\text{prior}}, \Sigma_t^{\text{prior}})$	$\mu_t^{\text{prior}} = A_t \mu_{t-1}$ $\Sigma_t^{\text{prior}} = A_t \Sigma_{t-1} A_t^\top + P_t$	$\eta_t^{\text{prior}} = \Lambda_t^{\text{prior}} A_t \Lambda_{t-1}^{-1} \eta_{t-1}$ $\Lambda_t^{\text{prior}} = (A_t \Lambda_{t-1}^{-1} A_t^\top + P_t)^{-1}$
Posterior / Update $z_t v_{1:t} \sim \mathcal{N}(\mu_t, \Sigma_t)$	$\mu_t = \mu_t^{\text{prior}} + K_t (v_t - C_t \mu_t^{\text{prior}})$ $\Sigma_t = (I - K_t C_t) \Sigma_t^{\text{prior}}$ $K_t = \Sigma_t^{\text{prior}} C_t^\top (C_t \Sigma_t^{\text{prior}} C_t^\top + (\Lambda_t^v)^{-1})^{-1}$	$\Lambda_t = \Lambda_t^{\text{prior}} + C_t^\top \Lambda_t^v C_t$ $\eta_t = \eta_t^{\text{prior}} + C_t^\top \Lambda_t^v v_t$

Table 6. Two equivalent views at time t : moment (KF) vs. canonical (IF). Superscript “prior” denotes the predictive prior; unadorned symbols denote the posterior.

Intuition on noise. In our linear-Gaussian state-space model, the “noise” terms w_t and v_t are not claims that language or the world is intrinsically random; they compactly represent *uncertainty* and unmodeled effects (domain shift, annotation/embedding error, missing context). Their covariances P_t and Σ_t^{obs} calibrate trust in dynamics vs. data-larger P_t yields more adaptivity (faster forgetting), while larger Σ_t^{obs} leans on the prior-producing principled “gates” via precision ratios. In contrast, ‘noise’ in diffusion models is an *algorithmic corruption* deliberately added in a forward process to define a denoising/score-matching objective; its schedule (β_t) is chosen for optimisation/SNR shaping rather than for model measurement uncertainty.

Method	Gated state update*	Gates from
<i>Correlation write into a (decayed) state</i>		
Linear Attn.	$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	-
Mamba-1 (S6)	$\mathbf{S}_t = \bar{\mathbf{A}}_t \odot \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	$\mathbf{A}, \bar{\mathbf{A}}_t$
Mamba-2	$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$	a, α_t
<i>Delta (residual) write</i>		
DeltaNet	$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$	β_t
Gated DeltaNet	$\mathbf{S}_t = \alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$	α_t, β_t
KDA (Team et al., 2025)	$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top$	$\mathbf{D}_t, \beta_t$
GDN-2 (Hatamizadeh et al., 2026)	$\mathbf{S}_t = (\mathbf{I} - \mathbf{k}_t (\mathbf{b}_t \mathbf{k}_t)^\top) \mathbf{D}_t \mathbf{S}_{t-1} + \mathbf{k}_t (\mathbf{w}_t \mathbf{v}_t)^\top$	$\mathbf{D}_t, \mathbf{b}_t, \mathbf{w}_t$
<i>Bayesian filtering write (gates from Bayes' rule)</i>		
KLA (Ours)	$\mathbf{S}_t = \bar{\mathbf{A}} \odot \left(\mathbf{1}\mathbf{1}^\top - \frac{\mathbf{k}_t^2 (\Lambda_t^v)^\top}{\Lambda_t} \right) \odot \mathbf{S}_{t-1} + \frac{\mathbf{k}_t (\Lambda_t^v \mathbf{v}_t)^\top}{\Lambda_t}$	$\bar{\mathbf{A}}, \bar{\mathbf{P}}, \Lambda_t^v$, Möbius-structured posterior Λ_t

Table 7. **KLA as a gated RNN among linear-attention / gated-RNN mixers.** Each row is a gated state recurrence $\mathbf{S}_t = (\text{forget gate}) \mathbf{S}_{t-1} + (\text{input gate}) \mathbf{k}_t \mathbf{v}_t^\top$: **orange** marks the forget gate (coefficient of $\mathbf{S}_{t-1}$) and **green** the input/write gate (coefficient of $\mathbf{k}_t \mathbf{v}_t^\top$). In Mamba/GLA and the delta-rule variants these gates are *free, ad-hoc* functions of the token; in KLA (**blue**) they are instead *computed* from the running posterior precision Λ_t via the Möbius recursion (Equation (22)) - i.e. they *emerge from Bayes' rule*. Subscript t denotes per-token learned quantities; symbols without subscript denote global parameters.

E. Gated-Update View: KLA among Linear-Attention and Gated-RNN Mixers

Many recent sub-quadratic mixers can be written as a single *gated* state recurrence $\mathbf{S}_t = (\text{forget}) \mathbf{S}_{t-1} + (\text{input}) \mathbf{k}_t \mathbf{v}_t^\top$ (Table 7); a complementary *online-learning* / fast-weight reading (Liu et al., 2025; Yang et al., 2023; 2024; Katharopoulos et al., 2020; Dao & Gu, 2024) views each update as the closed-form minimiser of a per-token objective that trades a *proximal* term (stay close to the decayed memory) against a *fit* term (absorb the current token evidence).

Reading the information-mean recursion of Theorem 2 in moment coordinates, $\mu_t = \eta_t \odot \lambda_t$, collapses the predict/update pair into a single gated linear recurrence-the diagonal/scalar instance of the KLA row of Table 7:

Corollary 3 (Posterior-mean recursion as a gated RNN update). *The moment-form posterior mean $\mu_t = \eta_t \odot \lambda_t$ of the information recursion in Theorem 2 satisfies the gated recurrence*

$$\mu_t = \bar{\mathbf{a}}_t \odot \left(\mathbf{1} - \frac{\mathbf{k}_t^2 \odot \Lambda_t^v}{\lambda_t} \right) \odot \mu_{t-1} + \frac{\mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t}{\lambda_t}, \quad (20)$$

the diagonal/scalar instance of the KLA state update in Table 7.

We prove Corollary 3 in Section F, using $\lambda_t^{\text{prior}} = \rho_t \odot \lambda_{t-1}$ from Step 1 of Theorem 1. This μ_t is the exact minimiser $\arg \min_{\mu} L_t$ of the precision-weighted least-squares objective

$$L_t(\mu) = \Lambda_t^v \|\mathbf{v}_t - \mathbf{k}_t \odot \mu\|^2 + \rho_t \odot \lambda_{t-1} \|\mu - \bar{\mathbf{a}}_t \odot \mu_{t-1}\|^2, \quad (21)$$

balancing a fit term (weighted by the observation precision Λ_t^v) against a proximal term anchoring μ to the propagated mean $\bar{\mathbf{a}}_t \odot \mu_{t-1}$ with weight the prior precision $\lambda_t^{\text{prior}} = \rho_t \odot \lambda_{t-1}$, the history-dependent factor ρ_t of Equation (16), i.e. the prior-to-previous precision ratio. Table 7 places KLA alongside representative delta-rule and gated mixers as a gated state recurrence; setting $\bar{\mathbf{p}}_t = 0$ at constant precision freezes this gate and recovers a Mamba-2-style fixed-decay recurrence.

*For two matrices $\mathbf{A}, \mathbf{B}$ of equal shape we overload the fraction $\mathbf{A}/\mathbf{B}$ to denote *element-wise* (Hadamard) division; products between vectors of equal dimension (e.g. $\mathbf{b}_t \mathbf{k}_t, \mathbf{w}_t \mathbf{v}_t, \Lambda_t^v \mathbf{v}_t$) are element-wise. Vector outer products such as $\mathbf{k}_t \mathbf{v}_t^\top$ and $\mathbf{k}_t^2 (\Lambda_t^v)^\top$ form $d_k \times d_v$ matrices in the standard way.

F. Theorems and Proofs

Theorem 1 (Precision Update as a Möbius Transformation). *Let λ_t be the posterior precision at time t in the diagonal linear-Gaussian model Equation (7)-Equation (9). Define $\phi_t := k_t^2 \odot \Lambda_t^v$. Then the map $\lambda_{t-1} \mapsto \lambda_t$ is a linear-fractional (Möbius) transform:*

$$\lambda_t = M_t(\lambda_{t-1}) = \frac{\alpha_t \odot \lambda_{t-1} + \beta_t}{\gamma_t \odot \lambda_{t-1} + \delta_t}, \quad (14)$$

$$M_t = \begin{pmatrix} \alpha_t & \beta_t \\ \gamma_t & \delta_t \end{pmatrix} = \begin{pmatrix} 1 + \bar{p}_t \odot \phi_t & \bar{a}_t^2 \odot \phi_t \\ \bar{p}_t & \bar{a}_t^2 \end{pmatrix}. \quad (15)$$

Proof of Theorem 1. Consider the 1D (or diagonal) linear-Gaussian state space model

$$z_t = a_t z_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, p_t), \quad v_t = k_t z_t + \nu_t, \quad \nu_t \sim \mathcal{N}(0, (\Lambda_t^v)^{-1}),$$

and define the posterior precision $\lambda_t := \text{Var}(z_t \mid v_{1:t})^{-1}$.

Step 1 (Information-form predict/update).

$$\text{(predict)} \quad \lambda_t^{\text{prior}} = (a_t^2 \lambda_{t-1}^{-1} + p_t)^{-1} = \frac{\lambda_{t-1}}{a_t^2 + p_t \lambda_{t-1}}, \quad \text{(update)} \quad \lambda_t = \lambda_t^{\text{prior}} + k_t^2 \Lambda_t^v.$$

Step 2 (Single-step recursion and rearrangement). Eliminate the intermediate λ_t^{prior} to obtain

$$\lambda_t = \frac{\lambda_{t-1}}{a_t^2 + p_t \lambda_{t-1}} + k_t^2 \Lambda_t^v = \frac{(1 + p_t k_t^2 \Lambda_t^v) \lambda_{t-1} + a_t^2 k_t^2 \Lambda_t^v}{p_t \lambda_{t-1} + a_t^2}.$$

Hence λ_t is a linear-fractional (Möbius) transform of λ_{t-1} ,

$$\lambda_t = M_t(\lambda_{t-1}) = \frac{\alpha_t \lambda_{t-1} + \beta_t}{\gamma_t \lambda_{t-1} + \delta_t}, \quad M_t = \begin{pmatrix} \alpha_t & \beta_t \\ \gamma_t & \delta_t \end{pmatrix} = \begin{pmatrix} 1 + p_t \phi_t & a_t^2 \phi_t \\ p_t & a_t^2 \end{pmatrix},$$

where $\phi_t := k_t^2 \Lambda_t^v$. Thus, the precision recursion is a Möbius transformation with the stated matrix form. The diagonal multivariate case follows by applying the same scalar derivation to each diagonal element. □

Corollary 1 (Precision Updates via Parallel Prefix Scan). *Given λ_0 and matrices $\{M_t\}_{t=1}^T$ from Theorem 1, the posterior precisions $\{\lambda_t\}_{t=1}^T$ can be computed by a parallel prefix scan over $\{M_t\}$ with $\mathcal{O}(T)$ work and $\mathcal{O}(\log T)$ depth.*

Proof of Corollary 1. By Theorem 1, each precision update is a Möbius transformation: $\lambda_t = M_t(\lambda_{t-1})$. Since Möbius transformations compose via matrix multiplication, the precision at time t can be expressed as

$$\lambda_t = M_t \circ M_{t-1} \circ \cdots \circ M_1(\lambda_0) = (M_t \cdot M_{t-1} \cdots M_1) \cdot \lambda_0,$$

where the composition $M_{1:t} := \prod_{s=1}^t M_s$ is computed via standard 2×2 matrix multiplication. Since matrix multiplication is associative, the product $M_{1:T}$ can be computed via a parallel prefix scan (Blelloch, 1990) with $\mathcal{O}(T)$ work and $\mathcal{O}(\log T)$ depth on T processors. Once the prefix products $\{M_{1:t}\}_{t=1}^T$ are available, each precision $\lambda_t = M_{1:t} \cdot \lambda_0$ is obtained in $\mathcal{O}(1)$ time, yielding the claimed complexity. □

Remark (Practical Implementation). *In practice, the parallel prefix scan can be implemented efficiently on modern hardware using frameworks such as JAX’s `lax.associative_scan` or PyTorch’s `parallel_primitives`. The $\mathcal{O}(\log T)$ depth translates to logarithmic wall-clock time on sufficiently parallel hardware, matching the computational efficiency of deterministic SSMS like Mamba while maintaining probabilistic semantics.*

Theorem 2 (Mean Update as Affine Transformations). *Let $\eta_t := \lambda_t \odot \mu_t$ be the posterior information mean and Λ_t^v the value precision. Given the precision path $\{\lambda_t\}$, the information mean evolves affinely:*

$$\eta_t = \underbrace{(\bar{\mathbf{a}}_t^2 + \bar{\mathbf{p}}_t \odot \lambda_{t-1})^{-1} \odot \bar{\mathbf{a}}_t}_{f_t \text{ (history-dependent forget gate)}} \odot \eta_{t-1} + \underbrace{\mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t}_{\text{token evidence}}. \quad (17)$$

Proof of Theorem 2. Write the information parameters as $\eta_t := \Lambda_t \mu_t$ and let Λ_t^v denote the value precision. The information-form measurement update is

$$\Lambda_t = \Lambda_t^{\text{prior}} + k^2 \Lambda_t^v, \quad \eta_t = \eta_t^{\text{prior}} + k \Lambda_t^v v_t,$$

so the observation contribution is affine with coefficient $k \Lambda_t^v$.

For the time-prediction of the information mean, use $\mu_t^{\text{prior}} = a \mu_{t-1}$ and $\eta_t^{\text{prior}} = \Lambda_t^{\text{prior}} \mu_t^{\text{prior}}$. Eliminating the means gives

$$\eta_t^{\text{prior}} = \Lambda_t^{\text{prior}} a \mu_{t-1} = \Lambda_t^{\text{prior}} a \Lambda_{t-1}^{-1} \eta_{t-1}.$$

Hence, given the (known) precision path $\{\Lambda_t, \Lambda_t^{\text{prior}}\}$,

$$\eta_t = \underbrace{(\Lambda_t^{\text{prior}} a \Lambda_{t-1}^{-1})}_{f_t} \eta_{t-1} + k \Lambda_t^v v_t, \quad \mu_t = \Lambda_t^{-1} \eta_t.$$

Finally, substituting the information-form predict step $\Lambda_t^{\text{prior}} = \Lambda_{t-1} / (a^2 + p \Lambda_{t-1})$ (Step 1 of the proof of Theorem 1) eliminates Λ_t^{prior} from the forget gate,

$$f_t = \Lambda_t^{\text{prior}} a \Lambda_{t-1}^{-1} = \frac{a}{a^2 + p \Lambda_{t-1}} = (a^2 + p \Lambda_{t-1})^{-1} a,$$

which is exactly the forget gate in the statement of Theorem 2. This proves the claimed affine form. $\square$

Corollary 3 (Posterior-mean recursion as a gated RNN update). *The moment-form posterior mean $\mu_t = \eta_t \odot \lambda_t$ of the information recursion in Theorem 2 satisfies the gated recurrence*

$$\mu_t = \bar{\mathbf{a}}_t \odot \left(\mathbf{1} - \frac{\mathbf{k}_t^2 \odot \Lambda_t^v}{\lambda_t} \right) \odot \mu_{t-1} + \frac{\mathbf{k}_t \odot \Lambda_t^v \odot \mathbf{v}_t}{\lambda_t}, \quad (20)$$

the diagonal/scalar instance of the KLA state update in Table 7.

Proof of Corollary 3. By Theorem 2, $\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^v v_t$ with forget gate $f_t = \lambda_t^{\text{prior}} a_t \lambda_{t-1}^{-1}$. By Step 1 of the proof of Theorem 1, the predicted precision is

$$\lambda_t^{\text{prior}} = \frac{\lambda_{t-1}}{a_t^2 + p_t \lambda_{t-1}} = \rho_t \lambda_{t-1}, \quad \lambda_t = \lambda_t^{\text{prior}} + k_t^2 \Lambda_t^v.$$

Substituting $\eta_{t-1} = \lambda_{t-1} \mu_{t-1}$ and $\mu_t = \eta_t / \lambda_t$, and using $f_t \lambda_{t-1} = \lambda_t^{\text{prior}} a_t$,

$$\mu_t = \frac{\lambda_t^{\text{prior}} a_t \mu_{t-1} + k_t \Lambda_t^v v_t}{\lambda_t}.$$

Since $\lambda_t^{\text{prior}} = \lambda_t - k_t^2 \Lambda_t^v$, we have $\lambda_t^{\text{prior}} / \lambda_t = 1 - k_t^2 \Lambda_t^v / \lambda_t$, hence

$$\mu_t = a_t \left(1 - \frac{k_t^2 \Lambda_t^v}{\lambda_t} \right) \mu_{t-1} + \frac{k_t \Lambda_t^v}{\lambda_t} v_t,$$

the claimed gated recurrence. The diagonal multivariate case applies elementwise. $\square$

Theorem 3 (Convolutional Form for Deterministic LTI Systems). *Under the conditions: (i) time-invariant dynamics, $\mathbf{a}_t \equiv \mathbf{a}$, $\mathbf{k}_t \equiv \mathbf{k}$; and (ii) deterministic dynamics, process noise $\mathbf{p}_t = \mathbf{0}$, both the precision and information mean updates reduce to block-Toeplitz convolutions computable in $\mathcal{O}(T \log T)$ time via FFT/NTT:*

Precision updates: $\Lambda_t = \sum_{s=0}^t \mathbf{a}^{-2(t-s)} \odot \mathbf{k}^2 \odot \Lambda_s^v$

Information mean updates: $\eta_t = \mathbf{k} \odot \sum_{s=0}^t \mathbf{a}^{t-s} \odot \Lambda_s^v \odot \mathbf{v}_s, \quad \mu_t = \Lambda_t^{-1} \odot \eta_t$

Proof of Theorem 3. When $p_t = 0$ (deterministic dynamics), the Möbius transformation from Theorem 1 simplifies. Specifically, with zero process noise, the precision recursion becomes $\lambda_t = a^{-2}\lambda_{t-1} + k^2\Lambda_t^v$. For time-invariant a and k , this unrolls to:

$$\lambda_t = a^{-2t}\lambda_0 + k^2 \sum_{s=0}^t a^{-2(t-s)}\Lambda_s^v$$

which is a discrete convolution with kernel $\kappa[n] = k^2 a^{-2n}$.

Similarly, from Theorem 2, when $p_t = 0$ we have $f_t = a^{-1}$ constant, so the information mean recursion $\eta_t = a^{-1}\eta_{t-1} + k\Lambda_t^v v_t$ unrolls to:

$$\eta_t = k \sum_{s=0}^t a^{t-s} \Lambda_s^v v_s$$

which is also a convolution with kernel $h[n] = k a^n$.

Both convolutions have Toeplitz structure and can be computed via FFT in $\mathcal{O}(\log T)$ parallel time using the convolution theorem. $\square$

Remark (Practical Note). *While theoretically interesting, the convolutional form is primarily relevant for perfectly deterministic systems ($\mathbf{p} = \mathbf{0}$). In practice, KLA uses the more general parallel scan formulation (Corollary 1) which handles stochastic dynamics ($\mathbf{p}_t > \mathbf{0}$) efficiently.*

Corollary 2 (Mean Updates via Parallel Prefix Scan). *The posterior information means $\{\eta_t\}_{t=1}^T$ are computable via parallel prefix scan over the affine transformations in Theorem 2 in $\mathcal{O}(T)$ work and $\mathcal{O}(\log T)$ depth.*

Proof of Corollary 2. From Theorem 2, the information mean update has the affine form:

$$\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^\vee v_t$$

where $f_t = \frac{a_t}{a_t^2 + p_t \lambda_{t-1}}$ (which simplifies to a^{-1} when $p_t = 0$).

Define the binary associative operator $\oplus$ on pairs (f, b) by:

$$(f_2, b_2) \oplus (f_1, b_1) = (f_2 f_1, f_2 b_1 + b_2)$$

This is the standard associative operator for affine transformations.

The composed transformation at time t is:

$$(f_{1:t}, b_{1:t}) = (f_t, k_t \Lambda_t^\vee v_t) \oplus (f_{t-1}, k_{t-1} \Lambda_{t-1}^\vee v_{t-1}) \oplus \cdots \oplus (f_1, k_1 \Lambda_1^\vee v_1)$$

which gives $\eta_t = f_{1:t} \eta_0 + b_{1:t}$.

Since $\oplus$ is associative, the sequence of compositions can be computed via a parallel prefix scan (Blelloch, 1990) with $\mathcal{O}(T)$ work and $\mathcal{O}(\log T)$ depth. Once all information means $\{\eta_t\}_{t=1}^T$ are computed, the posterior means are obtained by normalising with the precision path from Corollary 1: $\mu_t = \Lambda_t^{-1} \eta_t$. $\square$

G. Probabilistic Interpretation and State-Expanded Recursions

This appendix expands the state expansion of Section 5.2: the probabilistic interpretation of the matrix-structured state (Section G.1), and the explicit state-expanded precision and mean recursions the scan computes (Section G.2).

G.1. Probabilistic interpretation

The state-expanded filter carries a matrix state whose entry $z^{(i,j)}$ is the i -th latent coordinate of value channel j ; it admits two equivalent readings. (i) *Factored state-space model*. Taken at face value, the outer-product construction of Section 5.2 is a *factored* SSM: each entry (i, j) carries its own scalar observation $v_t^{(j)} = k_t^{(i)} z_t^{(i,j)} + \varepsilon_t^{(i,j)}$ with $\varepsilon_t^{(i,j)} \sim \mathcal{N}(0, (\Lambda_t^v)^{-1})$ and runs an *independent* scalar Kalman filter (Theorems 1 and 2) - ND decoupled filters sharing only the data-dependent drive $\mathbf{k}_t \mathbf{v}_t^\top$. (ii) *Mean-field approximation*. Equivalently, this diagonal filter is the *mean-field* approximation of the exact filter of the natural, *coupled* model $\mathbf{v}_t = \mathbf{S}_t^\top \mathbf{k}_t + \varepsilon$, whose information-form measurement update adds the *full* rank-one key Gram $\Lambda_t^v \mathbf{k}_t \mathbf{k}_t^\top$; its off-diagonals $k_t^{(i)} k_t^{(i')}$ couple the N key slots and make the posterior precision a full $N \times N$ matrix, whereas KLA keeps only the diagonal $\Lambda_t^v \mathbf{k}_t^2$ and decouples the filter. The two readings *coincide* - identical posterior mean, hence identical layer output - and differ only in the discarded posterior covariance, so the shipped (mean-only) recursion is the same under either interpretation. The price of the diagonal choice is a Hebbian contraction readout rather than the *whitening* (pseudoinverse) associative memory the full precision would give; recovering the latter through a full-covariance filter is a natural direction for future work.

G.2. State-expanded precision and mean recursions

The Mobius-Scan and Linear-Scan steps of Algorithm 1 are the state-expanded ($N > 1$) matrix versions of the per-channel recursions Equations (14) and (17). Adopting the column-vector convention of Table 2 ($\mathbf{k}_t \in \mathbb{R}^N$, $\mathbf{v}_t, \Lambda_t^v \in \mathbb{R}^D$, information-mean state $\mathbf{H}_t = \boldsymbol{\eta}_t$ and precision Λ_t in $\mathbb{R}^{N \times D}$), they read

$$\Lambda_t = \frac{(\mathbf{1} + \bar{\mathbf{P}} \odot \Phi_t) \odot \Lambda_{t-1} + \bar{\mathbf{A}}^2 \odot \Phi_t}{\bar{\mathbf{P}} \odot \Lambda_{t-1} + \bar{\mathbf{A}}^2}, \quad \Phi_t = \mathbf{k}_t^2 (\Lambda_t^v)^\top, \quad (22)$$

$$\mathbf{H}_t = \mathbf{F}_t \odot \mathbf{H}_{t-1} + \mathbf{k}_t (\Lambda_t^v \odot \mathbf{v}_t)^\top, \quad \mathbf{F}_t = \bar{\mathbf{A}} \odot (\bar{\mathbf{A}}^2 + \bar{\mathbf{P}} \odot \Lambda_{t-1}), \quad (23)$$

where $\odot, \otimes$ act elementwise on $N \times D$ arrays, and the rank-one outer products $\mathbf{k}_t^2 (\Lambda_t^v)^\top$ and $\mathbf{k}_t (\Lambda_t^v \odot \mathbf{v}_t)^\top$ supply the per-slot evidence (mirroring the $\mathbf{k}_t \mathbf{v}_t^\top$ term of Table 2). Here $\mathbf{H}_t = \boldsymbol{\eta}_t$ is the *information* (natural-parameter) mean; the *moment-form* state of Table 2 is $\mathbf{S}_t = \mathbf{H}_t \otimes \Lambda_t$ (i.e. $\boldsymbol{\mu}_t = \boldsymbol{\eta}_t \otimes \boldsymbol{\lambda}_t$), and the posterior mean reads out as $\mathbf{y}_t = \mathbf{q}_t \mathbf{S}_t$. Setting $N=1$ recovers the scalar recursions Equations (14) and (17).

-appendices continue on next page-

H. Additional Empirical Results

H.1. Ablation on the Importance of Process Noise Parameter

We conduct an ablation that isolates the process-noise parameter by fixing $\mathbf{p}_t = 0$ (deterministic dynamics) while leaving the rest of KLA unchanged - in particular its *adaptive*, input-dependent observation operator $\mathbf{k}_t$. Setting $\mathbf{p}_t = 0$ freezes the state-dependent factor ρ_t in the Möbius precision recurrence to a constant and collapses it to a fixed-gate linear update; the recurrence is still a parallel prefix scan with $\mathcal{O}(T)$ work and $\mathcal{O}(\log T)$ depth, but its gating is no longer history-dependent. (The stronger convolutional form of Theorem 3, computable via FFT, requires *both* $\mathbf{p}_t = 0$ and a time-invariant observation operator $\mathbf{k}_t \equiv \mathbf{k}$; with the adaptive $\mathbf{k}_t$ retained here the ablated system is linear but not time-invariant.) This ablation investigates whether the nonlinear gating supplied by process noise translates into practical benefits.

Figure 7 compares full KLA (learnable, time-invariant process noise $\mathbf{p}$) against a deterministic variant with process noise fixed to zero ($\mathbf{p}_t=0$, but retaining observation variance $(\Lambda^\vee)^{-1}$), which collapses the Möbius precision recurrence to a fixed-gate linear update. In the single-block MAD-Lab setting the memorisation and recall tasks are essentially saturated for every architecture and show little cross-method variance (Figure 4b); the skills that actually discriminate between models are Compression, Fuzzy Recall, and Selective Copy. Turning process noise off degrades exactly these three hardest tasks, while leaving memorisation, context recall, and noisy recall at $\sim 100\%$: Selective Copy and Compression each fall by more than 10 points (-14.9 and -12.1 , respectively) and Fuzzy Recall by 2.5, for a 4.7-point average drop.

These drops move KLA from best-in-class to near the bottom of the field on the very skill it is designed for: on Selective Copy the deterministic variant (75.7) trails every baseline except mLSTM (Figure 4b), erasing KLA’s selection-under-uncertainty advantage. The effect is more pronounced at the larger pretraining scale of Section 6.5, with longer contexts and multiple stacked KLA blocks: there, fixing $\mathbf{p}_t=0$ frequently destabilises training and diverges to NaNs, as the precision recursion degenerates into a linear scan in which the posterior precision λ_t grows unbounded and the nonlinear, history-dependent gating (Equation (16)) is lost; a non-zero process noise caps the accumulated precision and restores a fading, well-conditioned memory. A theoretical investigation of the stability properties of the system, particularly regarding Riccati controllability and observability, and the role of $\mathbf{p}_t$ in system stabilisation, represents an interesting direction for future work.

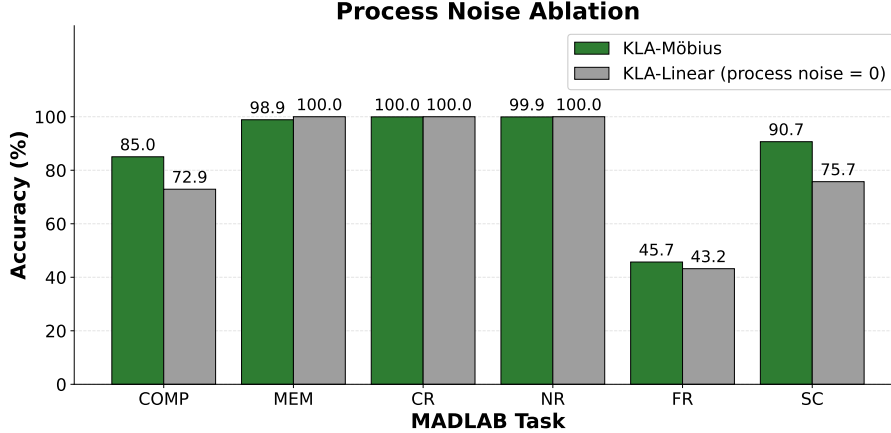


Figure 7. **Process-noise ablation on the MAD-Lab suite.** Full KLA (learnable, time-invariant process noise $\mathbf{p}$) versus a deterministic variant with $\mathbf{p}_t=0$, which collapses the Möbius precision recurrence to a fixed-gate linear update. With-noise baselines are the single-block MAD results of Figure 4b. Removing process noise leaves memorisation and exact recall unchanged but degrades the selection-heavy tasks (Selective Copy, Compression), for a 4.7-point average drop.

H.2. Long-Context Associative Recall (Full Sweep)

Multi-Query Associative Recall (MQAR) (Arora et al., 2023) is particularly demanding for fixed-size recurrent architectures, as it directly tests their capacity to store and retrieve multiple key-value associations, a fundamental bottleneck often limited by state dimensionality. Following the Zoology benchmark (Arora et al., 2023), we evaluate on a considerably harder configuration than typically studied: sequence length $T = 2048$ with vocabulary size $V = 256$ (matching or exceeding model dimensions; see Table 10).

Figure 8 reveals distinct scaling behaviours across architectures. KLA consistently outperforms Mamba across all dimensions

and substantially outperforms GLA, which fails to learn the task under this extreme setting. At lower dimensions ($d = 64, 128$), GDN performs best; its delta-rule mechanism is specifically designed for strong associative recall at limited capacity (Yang et al., 2024), and it operates with a slightly higher effective state size (see Table 13). At $d = 256$, KLA outperforms all baselines and reaches near-perfect accuracy ($> 95\%$).

We hypothesise that uncertainty-weighted updates help mitigate state “saturation” under heavy key-value load. Unlike purely additive accumulators, KLA’s fractional-linear (Möbius) precision updates (Equation (16)) adaptively down-weight observations based on posterior uncertainty: when the latent state becomes saturated with many associations, high-confidence historical information naturally suppresses unreliable new observations. This provides an implicit, learned compression mechanism, analogous to selective pattern storage in high-capacity associative memories (Storkey & Valabregue, 1997), without requiring explicit dimension scaling.

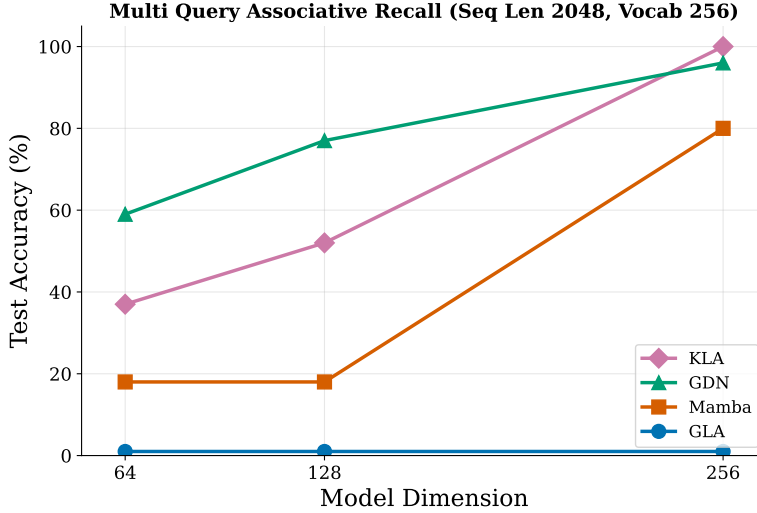


Figure 8. MQAR accuracy vs. model dimension ($T = 2048, V = 256$). KLA achieves near-perfect accuracy at dimension 256, outperforming GDN and Mamba. GLA fails to learn the task in this challenging setting.

H.3. Runtime Scaling

Setup. We benchmark forward-only runtimes for three KLA implementations: a recurrent (time-stepped) Kalman update, the built-in Torch associative scan, and a custom Triton associative scan kernel. All measurements use Torch 2.9.1 on an NVIDIA A100 (80GB), hidden size 960, and float32 precision. The forward-only pass uses a batch size of 1, simulating test-time prompt processing for a 182 million parameter model-critical for deploying sequence models in production settings.

Results. Figure 9 shows that the scan-based implementations scale efficiently with sequence length compared to the recurrent counterparts. These results confirm that information-form filtering can be implemented with the same scan-parallel profile as modern SSM/GLA-style mixers, making KLA practical for long-context prompt processing.

H.4. Equivalent Attention Matrix

As shown in Theorem 2, the information-mean recurrence $\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^v v_t$ (Equation (17)) can be unrolled into a lower-triangular attention matrix $\mathbf{W}$ whose entries are products of **history-dependent forget gates** f_s , keys k_j , and **observation precisions** Λ_j^v . Folding in the output readout query $\mathbf{q}$ and posterior precision scaling λ^{-1} gives the full per-channel sequence transformation $\mathbf{M}_{\text{seq}} = \text{diag}(\mathbf{q} \odot \lambda^{-1}) \mathbf{W}$, so that $\mathbf{y} = \mathbf{M}_{\text{seq}} \mathbf{v}$ (plus init-state terms). Figure 10 visualises this two-step structure.

H.5. Kalman Attention Map Visualisation

Unrolling the information-mean recurrence equation 17 yields the lower-triangular matrix $\mathbf{W}$ described in Section H.4; folding in the readout query $\mathbf{q}_t$ and precision scaling λ^{-1} gives the per-channel sequence transformation $\mathbf{M}_{\text{seq}} = \text{diag}(\mathbf{q} \odot \lambda^{-1}) \mathbf{W}$ (cf. Figure 10). Because $\mathbf{M}_{\text{seq}}$ is lower-triangular, it has the same causal structure as a standard attention matrix; we therefore call it the *Kalman Attention Matrix*.

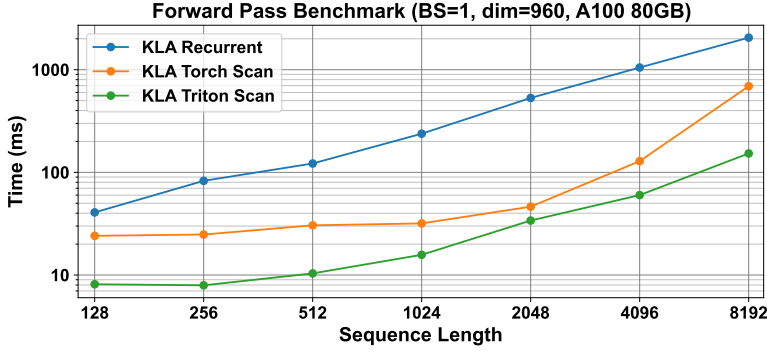


Figure 9. **Forward-only (prompt processing) runtime scaling.** Wall-clock runtime of KLA implementations across sequence lengths during the forward-only pass. Torch Scan uses `torch._higher_order_ops.associative_scan`; Triton Scan uses custom forward kernels.

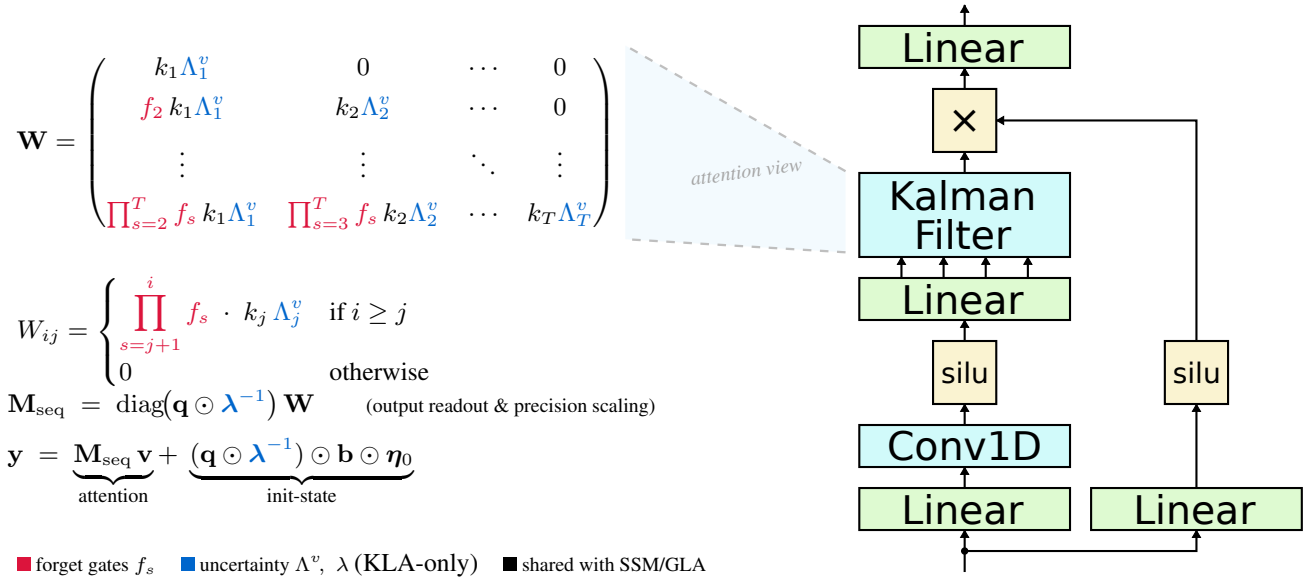


Figure 10. **Block architecture and equivalent attention form.** *Left:* The block architecture follows the fused-MLP design of Mamba, with the Kalman Filter as a drop-in replacement for any SSM/Attention primitive. *Right:* Unrolling the information-mean recurrence (Equation (17): $\eta_t = f_t \eta_{t-1} + k_t \Lambda_t^v v_t$) yields a lower-triangular matrix $\mathbf{W}$ whose entries are products of **history-dependent forget gates** f_s , keys k_j , and **observation precisions** Λ_j^v . Applying the output readout query q_i and **posterior precision** λ_i^{-1} gives the full sequence transformation $\mathbf{M}_{\text{seq}} = \text{diag}(\mathbf{q} \odot \boldsymbol{\lambda}^{-1}) \mathbf{W}$, the precision terms being unique to KLA and absent in standard SSMs/GLA.

Figures 11 to 13 visualise the Kalman attention patterns learned by KLA on three MADLAB tasks, each showing four randomly chosen channels. The attention weight at position (t, s) reflects how much the posterior mean at step t is influenced by the observation at step s , weighted by the learned precision ratio.

These visualisations demonstrate that KLA learns task-appropriate attention patterns: selective retrieval for copying tasks, channel specialisation for recall, and uniform retention for memorization-all while maintaining the causal structure inherent to the Kalman filter formulation.

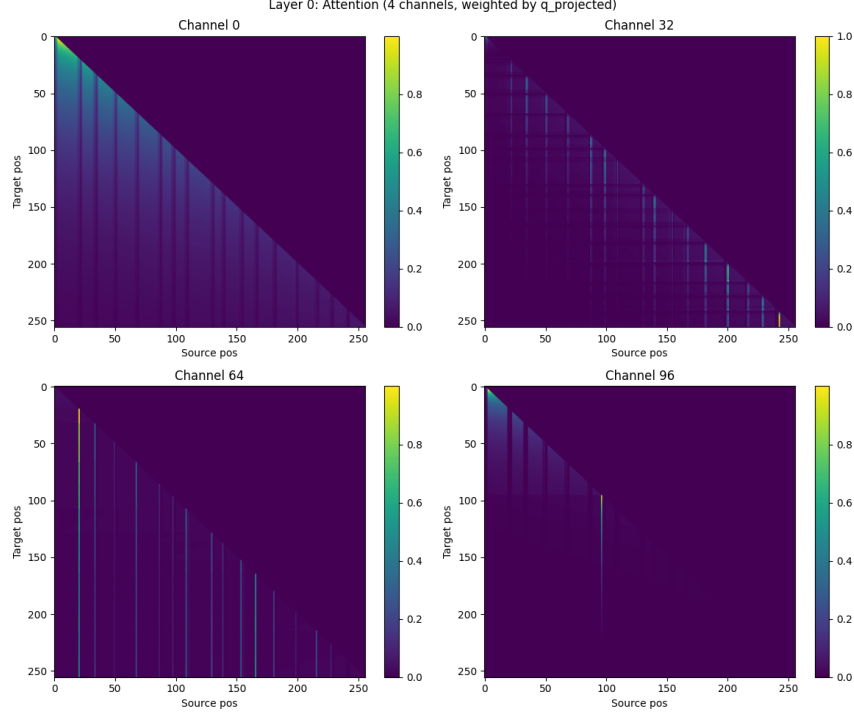


Figure 11. Attention maps for **Selective Copying** (sequence length 256). The model learns sparse, intermittent vertical bands, attending strongly to a small set of task-relevant positions (mostly copy positions). They either activate or suppress these relevant positions. Differences across channels indicate channel-level specialisation in which positions are retrieved.

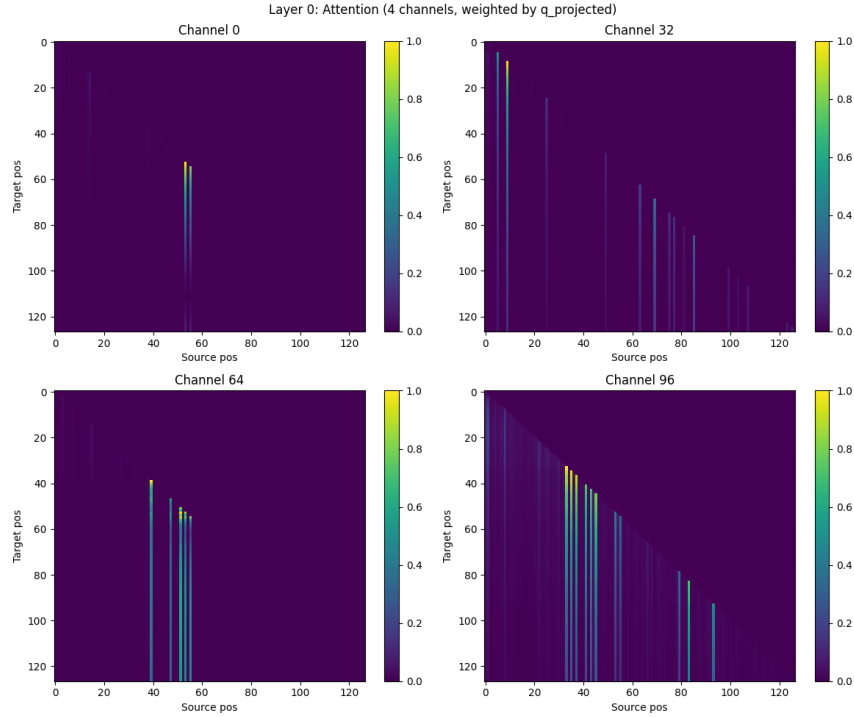


Figure 12. Attention maps for **In-Context Recall** (sequence length 128). The patterns are sparse and “pointer-like”: within each channel, attention concentrates on a small set of source positions (vertical bands) across many target steps, with a mostly low-activation background. Channels show specialisation trends.

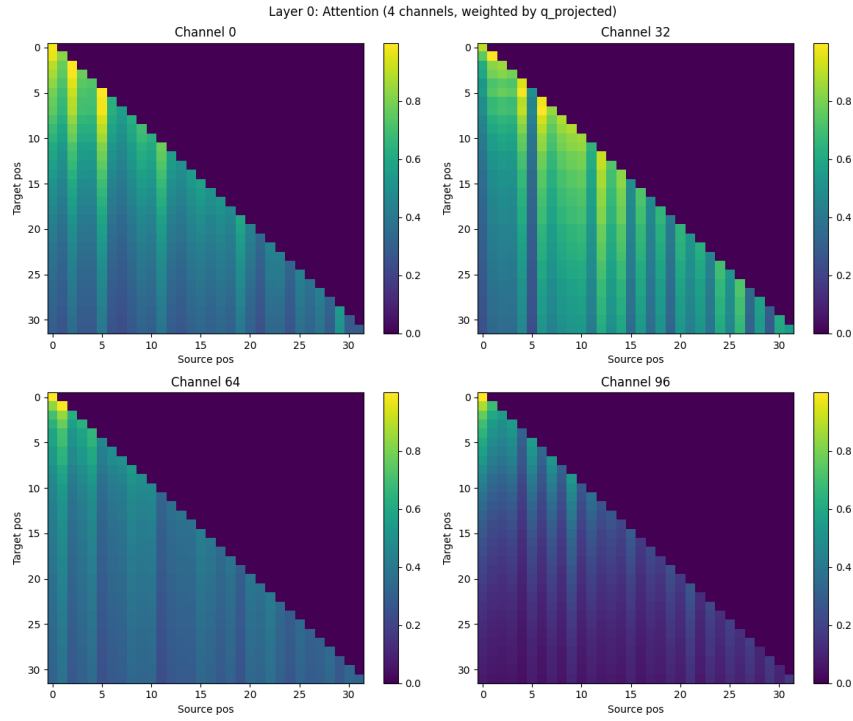


Figure 13. Attention maps for **Memorization** (sequence length 32). The smoother, more gradual decay reflects the task’s requirement to maintain information uniformly across the sequence. All channels show similar patterns, indicating that memorization benefits from redundant, distributed storage rather than selective attention.

Table 8. Mapping from language skills/capabilities to the synthetic probe tasks. Green ✓ indicates the primary probes/tasks for these skills.

Language Skill / Capability	In-context (MQAR)	Fuzzy Recall	Noisy Recall	Selective Copy	Compression	Memorization
Prompt-level associative retrieval & in-context learning (bindings, multi-query, remapping, basic composition)	✓	✓	✓	✓	✗	✗
Selective attention & noise robustness (noise filtering, long-context stability, calibration)	✗	✗	✓	✓	✗	✗
Ordered copying & pointer-like control flow (order-sensitive working memory)	✗	✗	✗	✓	✗	✗
Information aggregation/bottlenecking (token concatenation for downstream decoding)	✗	✗	✗	✗	✓	✗
Parametric memory of stable facts (weight-based knowledge)	✗	✗	✗	✗	✗	✓

I. Datasets and Benchmarks Used

I.1. MAD LM Suite

To evaluate KLA as a language-modeling primitive, we use a standardised suite of six discrete token manipulation tasks. We adopt the MAD framework (Poli et al., 2024), which consolidates synthetic tasks from prior work into a common specification of data generation, splits, and evaluation. Each task probes a distinct capability (associative retrieval, span compositionality, noise robustness, ordered copying, single-token information aggregation, and parametric memory), allowing us to disentangle model strengths and weaknesses across language skills (see Table 8).

We use the baseline configuration from the MAD framework for all six tasks. Table 9 summarises the key parameters for each task.

Table 9. Baseline configurations for the six MAD tasks used in our experiments.

Parameter	In-context (MQAR)	Fuzzy Recall	Noisy Recall	Selective Copy	Compression	Memorization
Vocab size	16	16	16	16	16	256
Training seqs	12,800	12,800	12,800	12,800	12,800	256
Sequence length	128	128	128	256	32	32
Noise tokens (%)	-	-	20%	-	-	-
Key/value split	8/8	8/8	8/8	-	-	128/128

Additional task-specific settings: Selective Copy (SC) uses `num_copy=16`; In-Context Recall (CR) and Fuzzy Recall (FR) use `multi_query=True`; FR uses motif sizes $k = v = 3$; Noisy Recall (NR) uses `noise_vocab=16` with noise fraction 0.2.

I.1.1. TASK DESCRIPTIONS

In-context recall (MQAR). Sequences consist of key-value pairs with separate vocabularies. The model must predict values for keys that appeared earlier in the sequence. Key-value mappings are randomly shuffled between sequences, forcing the model to learn in-context rather than memorising fixed associations.

Fuzzy in-context recall. An extension of in-context recall where keys and values are represented by variable-length spans (1-3 tokens). This tests the model’s ability to handle compositional keys and maintain associations across multi-token representations.

Noisy in-context recall. Similar to in-context recall, but with 20% of tokens from a separate noise vocabulary randomly inserted. This evaluates the model’s robustness to irrelevant information and selective attention capabilities.

Selective copying. Sequences contain random tokens interspersed with special [blank] and [insert] tokens. The model must copy non-special tokens to [insert] positions in order, while learning to ignore tokens near [blank] markers. This tests selective memorisation and order-preserving working memory.

Compression. Sequences of random tokens ending with a compression token [c]. The model must compress all sequence information into the representation at the compression token position, such that a fixed two-layer MLP can reconstruct any

input token given the compressed representation plus a positional encoding.

Memorization. A fixed key-value dictionary is used across all sequences. In each sequence, keys appear with their values masked by [insert] tokens. The model must learn the fixed mappings from the training data, as values never appear in the input during training.

I.1.2. STANDARD TRAINING SETUP IN MAD FRAMEWORK

All tasks use the same architectural scaffolding and training procedure. Training hyperparameters are detailed in Section J.

For most tasks, the training objective is standard next-token prediction via log likelihood (cross-entropy loss). The **Compression** task uses a different setup: the model’s output at the compression token position is passed through a separate two-layer MLP decoder (dimensions [240, 120]) along with a positional encoding to reconstruct each token in the sequence. During training, the model learns to compress the entire sequence into a single representation that enables accurate reconstruction.

All models use a single-layer architecture with a model dimension $d_{\text{model}} = 128$ and an effective state size of 2048.

I.2. Long-Context MQAR

Table 10 lists the data configuration for the long-context MQAR benchmark task (Arora et al., 2023).

Table 10. Long-Context MQAR data parameters.

Long-Context Multi-Query Associative Recall (MQAR)				
Setting	Seq. length	Vocab size	Training seqs	Eval seqs
CR (hard)	2,048	256	12,800	1,280

J. Implementation Details

J.1. Experimental Protocol

Statistical significance. We report results averaged over 5 random seeds to ensure statistical reliability. All comparisons between models use the same random seeds to ensure fairness. For all tasks, we report the mean performance across seeds. For the A5 state tracking task, we consider a task solved if the model achieves $\geq 90\%$ accuracy in at least one of the five seeds.

Open-source code. The complete implementation, including all custom Triton and CUDA kernels and experimental code, is publicly available at <https://github.com/vaisakh-shaj/kalman-linear-attention> to facilitate reproducibility and future research.

J.2. Training Hyperparameters

We use default settings unless specifically stated otherwise. Table 11 lists the training configuration used across all experiments.

Table 11. **Training Hyperparameters** (fixed across all models and tasks)

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1×10^{-3}
Learning rate schedule	None
Maximum epochs	750
Early stopping patience	70 epochs
Gradient clipping	5.0
Weight decay	0.0
Precision	32-bit float

KLA-specific settings: We use encoder MLP hidden dimension = 120, decoder MLP dimensions = [240, 120], process noise scale $\mathbf{p} = 0.01$ (initial value), discretisation timestep range $\Delta_t \in [0.001, 0.1]$, causal convolution kernel size = 4, and Monte Carlo samples = 10 (used for KLA probabilistic decoding; see Figure 4b).

J.3. MAD-Lab Hyperparameters

To ensure fair comparison, all models are configured with equal effective state sizes by adjusting architecture-specific expansion parameters. For each model dimension, we match the total number of parameters used for state representation across architectures. Note that GatedDeltaNet does not allow exact state-size matching due to constraints on the minimum number of heads and the requirement that expanded key and value dimensions be multiples of 16 or 32. Hence, we assign GatedDeltaNet the next-largest feasible value compared to the other baselines. Table 12 shows the dimensional configuration for each architecture.

Table 12. **Model Dimensions** (configured for equal state size $S = 2048$)

Model	# Blocks	d_{model}	d_{state}	Other	S
KLA (Ours)	1	128	8	expand=1	2048
Mamba	1	128	16	expand=1	2048
GatedDeltaNet	1	128	-	$e_k=0.5, e_v=1.0, H=4$	2048
GLA	1	128	-	$e_k=0.5, e_v=1.0, H=4$	2048
mLSTM	1	128	-	$H=16, \text{proj_factor}=2.0$	2048

Batch size: We use a batch size of 172 for all MAD-Lab experiments.

J.4. MQAR Hyperparameters

For the MQAR (hard) experiments, we evaluate across three model dimensions with equal state sizes per dimension. All models use 2 layers (repeated blocks). As in the MAD-Lab setup, GatedDeltaNet is given the nearest feasible configuration (see above). Table 13 lists the configuration for each architecture at each dimension.

Table 13. MQAR Model Configurations (state-matched per dimension)

Model	# Blocks	d_{model}	d_{state}	Other	State Size
$d = 64$ (batch size = 64)					
KLA	2	64	16	expand=1	2,048
Mamba	2	64	16	expand=2	2,048
GatedDeltaNet	2	64	-	$e_k=0.75, e_v=1.5, H=2$	2,304
GLA	2	64	-	$e_k=1.0, e_v=1.0, H=4$	2,048
$d = 128$ (batch size = 32)					
KLA	2	128	16	expand=1	4,096
Mamba	2	128	16	expand=2	4,096
GatedDeltaNet	2	128	-	$e_k=0.75, e_v=1.5, H=4$	4,608
GLA	2	128	-	$e_k=0.5, e_v=1.0, H=4$	4,096
$d = 256$ (batch size = 16)					
KLA	2	256	16	expand=1	8,192
Mamba	2	256	16	expand=2	8,192
GatedDeltaNet	2	256	-	$e_k=0.75, e_v=1.5, H=8$	9,216
GLA	2	256	-	$e_k=0.25, e_v=1.0, H=4$	8,192

J.5. A5 State Tracking Hyperparameters

For the A5 state tracking experiments (Section 6.4), we use $d_{\text{model}} = 1024$ and $d_{\text{state}} = 16$ for KLA. We train for up to 500 epochs with early stopping (patience 50), and a learning rate of 3×10^{-4} , and we evaluate over 5 seeds. Values for the baselines are reported as in Merrill et al. (2024).

J.6. Language-Modeling Pretraining and Evaluation Details

This appendix gives the pretraining and evaluation details for the language-modeling experiments of Section 6.5 (results in Table 3).

Compute. All language-modeling models were trained on NVIDIA H200 GPUs on the Isambard-AI National AI Research Resource (AIRR) (McIntosh-Smith et al., 2024).

Pretraining corpus and objective. All models are pretrained on FineWeb-Edu (Penedo et al., 2024) with the standard next-token-prediction (cross-entropy) objective in mixed precision (bfloat16, with the parallel scan layers kept in fp32). We train at two academic scales: a 45M-parameter model on 1.8B tokens and a 180M-parameter model on 10.9B tokens.

Models and baselines. Both the 45M and 180M models use a 12-layer backbone, and all architectures are parameter-matched at each scale. We compare four standalone mixers — GPT (softmax attention), Mamba, Gated DeltaNet (GDN), and KLA— and the hybrid GPT+KLA, in which *only the final attention layer* of the GPT backbone is replaced by a KLA block (Section 6.5). Baseline state/expansion settings are matched following the conventions in Section J.3; we report the KLA-specific per-scale configuration in Table 14.

Optimisation. Models are trained with AdamW and gradient clipping 3.0 (Section 6.6). The remaining optimisation settings — learning rate, schedule, warmup, batch size, and context length — are summarised in Table 14.

Zero-shot evaluation. We report zero-shot performance with the LM Evaluation Harness on eight commonsense benchmarks: LAMBADA (OpenAI split), HellaSwag, PIQA, ARC-Easy, ARC-Challenge, WinoGrande, OpenBookQA, and BoolQ (Table 3). We report accuracy (acc) for all tasks except HellaSwag and ARC-Challenge, for which we report length-normalised accuracy (acc_n).

Table 14. KLA language-modeling pretraining configurations. We report KLA-specific settings only. Both scales use a 12-layer KLA backbone. Baseline configurations are state-matched following Section J.3. All parameters are optimised with AdamW ($\beta = (0.8, 0.95)$, $\epsilon = 10^{-10}$); Muon is disabled. The per-group AdamW learning rate is scaled internally by $\sqrt{768/d_{\text{model}}} \cdot \sqrt{B/2^{19}}$ (where B is the token batch size); embeddings use a separate base rate of 0.3.

KLA hyperparameter	45M	180M
Layers (depth)	12	12
d_{model}	496	1360
d_{state}	32	32
Causal conv. kernel size	4	4
Process-noise init $\mathbf{p}$	0.01	0.01
Discretisation range Δ_t	[0.001, 0.1]	[0.001, 0.1]
Context length	2048	2048
Training tokens	1.8B	10.9B
Optimiser	AdamW	AdamW
Base learning rate (hidden/SSM)	4×10^{-3}	4×10^{-3}
Effective peak LR (hidden/SSM)	2.5×10^{-3}	3.0×10^{-3}
Embedding learning rate (base)	0.3	0.3
LR schedule	constant then linear decay (no warmup)	constant then linear decay (no warmup)
Warmup steps	0	0
Warmdown fraction	0.4	0.4
Weight decay (2D hidden/lm_head)	0.1	0.1
Weight decay (embed/1D)	0.0	0.0
Global batch size (tokens)	131,072	524,288
Gradient clipping	3.0	3.0
Precision	mixed bf16 (scan in fp32)	mixed bf16 (scan in fp32)